VŠB TECHNICKÁ UNIVERZITA OSTRAVA
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Multisenzorický měřící modul s protokolem Spinel

Multisensoric Measuring Module with Spinel Protocol

Pavel Valošek

Bakalářská práce

Vedoucí práce: Ing. Petr Olivka, Ph.D.

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/25/147407
vsboee25043BDC

Zadání bakalářské práce

Student: **Pavel Valošek**

Studijní program: B0613A140014 Informatika

Téma: Multisenzorický měřící modul s protokolem Spinel
Multisensoric Measuring Module with Spinel Protocol

Jazyk vypracování: čeština

Zásady pro vypracování:

Navrhněte multisenzorický měřící modul, které bude s nadřazeným počítačem komunikovat prostřednictvím sériové sběrnice RS485 a bude využívat protokol Spinel.
Použitý mikropočítač musí být 32bitový s jádrem ARM s podporou FreeRTOS. Pro měření budou využity senzory s rozhraním I2C.

1. Seznamte se s protokolem Spinel, jeho třemi vrstvami a podporovanými textovými a binárními formáty protokolu A, B i a.
2. Navrhněte koncepci programového řešení pro zpracování protokolu Spinel na jednotlivých vrstvách tak, aby bylo řešení použitelné na straně mikropočítače i v nadřazeném počítači.
3. Navrhněte nad protokolem Spinel modul měřícího zařízení tak, aby bylo možno připojit více teplotních senzorů TMP275 s rozhraním I2C, případně i na více rozhraní I2C.
4. Seznamte se s mikropočítačovým modulem FRDM-MCXA153 a vývojovým prostředím MCUXpresso.
5. Implementujte ve vybraném mikropočítači komunikaci s nadřazeným počítačem prostřednictvím rozhraní RS485 i s jednotlivými senzory jako samostatné úlohy FreeRTOSu.
6. Zhodnocení: stabilita, spolehlivost, modulárnost. Výsledky testů.

Seznam doporučené odborné literatury:

[1] Obecný popis protokolu Spinel, https://cdn.papouch.com/data/user-content/old_eshop/files_article/Spinel/spinel_obecne.pdf
[2] Popis implementace protokolu Spinel, https://cdn.papouch.com/data/user-content/old_eshop/files_article/Spinel/spinel_implementace.pdf
[3] Mikropočítačový modul FRDM-MCXA153, http://nxp.com/frdm-mcxa153
[4] FreeRTOS, Operační systém v reálném čase, https://www.freertos.org/
[5] Teplotní sensor TMP275, https://www.ti.com/lit/gpn/TMP275-Q1

17. listopadu 2172/15
708 00 Ostrava-Poruba
Česká republika

spojovatelka: +420 597 321 111
epodatelna: epodatelna@vsb.cz
ID datové schránky: d3kj88v

IČ: 61989100
DIČ: CZ61989100

email: studijni.fei@vsb.cz
www.fei.vsb.cz

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Petr Olivka, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 21.10.2025 11:00:23

Abstrakt

Bakalářská práce se zabývá návrhem a implementací multisenzorického měřicího modulu pro měření teploty, který komunikuje s nadřazeným počítačem prostřednictvím protokolu Spinel po sběrnici RS485. Řešení je realizováno na mikrokontroléru s jádrem ARM Cortex-M33 s využitím operačního systému FreeRTOS a teplotních senzorů TMP275 připojených přes sběrnici I^2C.

Součástí práce je rozbor protokolu Spinel, návrh jeho softwarové architektury a implementace sdílených částí protokolu použitelných jak na straně mikrokontroléru, tak na straně nadřazeného počítače. Dále je popsán návrh hardwarového řešení měřicího modulu, implementace aplikační vrstvy firmware a testovací aplikace pro osobní počítač.

Výsledkem práce je funkční prototyp měřicího modulu schopný detekovat připojené senzory, periodicky měřit teplotu, zpracovávat požadavky přenášené protokolem Spinel a ukládat vybrané konfigurační údaje do interní paměti Flash. Funkce navrženého řešení byla ověřena praktickým testováním komunikace, měření teploty i práce s konfiguračními údaji.

Klíčová slova

Spinel, RS485, FreeRTOS, ARM Cortex-M33, TMP275, I^2C, měřicí modul, vestavěný systém

Abstract

This bachelor thesis deals with the design and implementation of a multisensor temperature measurement module communicating with a host computer via the Spinel protocol over the RS485 bus. The solution is implemented on microcontroller with an ARM Cortex-M33 microcontroller using the FreeRTOS operating system and TMP275 temperature sensors connected via the I^2C bus.

The thesis includes an analysis of the Spinel protocol, the design of its software architecture, and the implementation of shared protocol components usable both on the microcontroller side and on the host computer side. It also describes the hardware design of the measurement module, the implementation of the firmware application layer, and a testing application for a personal computer.

The result of the thesis is a functional prototype of a measurement module capable of detecting connected sensors, periodically measuring temperature, processing requests transmitted by the Spinel protocol, and storing selected configuration parameters in internal Flash memory. The functionality of the proposed solution was verified by practical testing of communication, temperature measurement, and configuration handling.

Keywords

Spinel, RS485, FreeRTOS, ARM Cortex-M33, TMP275, I^2C, measurement module, embedded system

Poděkování

Rád bych poděkoval vedoucímu práce Petru Olivkovi za odborné vedení, ochotu při konzultacích, cenné připomínky a pomoc při řešení problémů v průběhu zpracování této bakalářské práce.

Obsah

Seznam použitých symbolů a zkratek **10**

Seznam obrázků **11**

Seznam tabulek **12**

1 Úvod **13**

2 Protokol Spinel **15**

2.1 Architektura protokolu 15

2.2 Struktura rámce 16

2.3 Formáty protokolu Spinel 16

2.4 Význam jednotlivých položek rámce 18

3 Návrh softwaru komunikačního protokolu **21**

3.1 Návrh protokolových vrstev 22

3.2 Správce protokolu 22

3.3 Zpracování příchozího rámce 22

3.4 Vrstva Frame 23

3.5 Vrstva SData 23

3.6 Shrnutí návrhu 23

4 Implementace komunikačního protokolu **24**

4.1 Implementace parseru 24

4.2 Implementace vrstvy Frame 26

4.3 Implementace vrstvy SData 27

4.4 Implementace správce protokolu 27

4.5 Shrnutí implementace 28

5 Návrh a realizace měřicího modulu **29**
5.1 Požadavky na měřicí modul 29
5.2 Vývojová deska FRDM-MCXN947 30
5.3 Převodník MAX485 a komunikace po sběrnici RS485 31
5.4 Sběrnice I^2C a připojení senzorů 32
5.5 Teplotní senzory TMP275 33
5.6 Celkové hardwarové zapojení 34
5.7 Shrnutí kapitoly 34

6 Implementace aplikační vrstvy a firmware měřicího modulu **36**
6.1 Struktura firmware měřicího modulu 36
6.2 Inicializace systému 37
6.3 Rozdělení programu do úloh FreeRTOS 37
6.4 Příjem a zpracování komunikačních rámců 37
6.5 Obsluha teplotních senzorů TMP275 38
6.6 Ukládání konfigurace do interní paměti Flash 38
6.7 Aplikační vrstva na straně mikrokontroléru 39
6.8 Aplikační vrstva na straně nadřazeného počítače 40
6.9 Shrnutí kapitoly 41

7 Ověření funkce navrženého řešení **42**
7.1 Postup oživování a ověřování systému 42
7.2 Ověření implementace protokolu Spinel 43
7.3 Ověření komunikace mikrokontroléru s nadřazeným počítačem 43
7.4 Ověření měření teploty pomocí senzoru TMP275 44
7.5 Ověření ukládání konfigurace do interní paměti Flash 45
7.6 Shrnutí kapitoly 45

8 Závěr **46**

Literatura **47**

Přílohy **47**

A Ukázky úspěšně provedených testů komunikace **48**
A.1 Čtení provozních a diagnostických údajů 48
A.2 Práce se stavovými a uživatelskými daty 49
A.3 Konfigurační a servisní instrukce 49

B Ukázky chybových a hraničních stavů komunikace **52**
B.1 Chybové odpovědi zařízení 52
B.2 Příliš dlouhý rámec 53
B.3 Další ošetřené hraniční stavy 53

Seznam použitých zkratek a symbolů

FreeRTOS	–	Free Real-Time Operating System
RISC	–	Reduced Instruction Set Computer
ARM	–	Advanced RISC Machine
RS485	–	Recommended Standard 485
I^2C	–	Inter-Integrated Circuit
UART	–	Universal Asynchronous Receiver-Transmitter
ASCII	–	American Standard Code for Information Interchange
MSN	–	Most Significant Nibble
LSN	–	Least Significant Nibble
MCU	–	Microcontroller Unit
SRAM	–	Static Random Access Memory
SPI	–	Serial Peripheral Interface
PLL	–	Phase-Locked Loop
USB	–	Universal Serial Bus
GND	–	Ground

Seznam obrázků

3.1 Vrstvené uspořádání softwarové implementace protokolu Spinel pro nadřazený počítač a mikrokontrolér 21

4.1 Stavový automat parseru protokolu Spinel 25

5.1 Vývojová deska FRDM-MCXN947 s vyznačenými hlavními částmi a typy konektorů. Převzato a upraveno podle [3] 30
5.2 Příklad úplného přenosu dat po sběrnici I^2C. Převzato z [6] 32
5.3 Blokové schéma hardwarového propojení nadřazeného počítače, měřicího modulu a teplotních senzorů TMP275 34
5.4 Laboratorní zapojení měřicího modulu použité při ověřování komunikace po sběrnici RS485 a měření teploty pomocí senzorů TMP275 35

7.1 Ukázka navázání komunikace testovací aplikace s měřicím modulem, detekce dostupných zařízení a zahájení periodického čtení teploty 44

A.1 Ukázka čtení provozních a diagnostických údajů zařízení 48
A.2 Ukázka čtení a zápisu stavového bajtu zařízení 49
A.3 Ukázka čtení a zápisu uživatelských dat zařízení 49
A.4 Ukázka čtení a změny nastavení kontrolního součtu 50
A.5 Ukázka resetu zařízení a následného ověření dostupných zařízení na lince 50
A.6 Ukázka povolení konfigurace, změny adresy zařízení a ověření nové adresy 51
A.7 Ukázka změny přenosové rychlosti a ověření nové hodnoty 51

B.1 Ukázka vybraných chybových stavů komunikace 52
B.2 Ukázka vybraných chybových stavů komunikace 53

Seznam tabulek

2.1 Struktura a popis ASCII rámce 16
2.2 Struktura a popis binárního rámce 16
2.3 Struktura dotazu ve formátu 65 17
2.4 Struktura odpovědi ve formátu 65 17
2.5 Struktura dotazu ve formátu 66 17
2.6 Struktura odpovědi ve formátu 66 17
2.7 Struktura dotazu ve formátu 97 18
2.8 Struktura odpovědi ve formátu 97 18
2.9 Kódy potvrzení ACK 20

6.1 Přehled implementovaných instrukcí aplikační vrstvy 40

Kapitola 1

Úvod

Cílem této bakalářské práce je návrh a implementace multisenzorického měřicího modulu pro měření teploty, který komunikuje s nadřazeným počítačem prostřednictvím protokolu Spinel přes sběrnici RS485. Práce se zaměřuje na využití moderního 32bitového mikrokontroléru s jádrem ARM a operačním systémem reálného času FreeRTOS, což umožňuje efektivně řídit paralelní úlohy a zajistit operace, které jsou časově kritické.

Měření teploty je jednou ze základních úloh průmyslových a laboratorních aplikací. Obvykle je potřeba snímat teplotu na více místech současně s dostatečnou přesností, stabilitou a spolehlivostí. Multisenzorické řešení umožňuje současné zpracování dat z více čidel připojených k jednomu měřicímu modulu. V navrhovaném systému budou využity teplotní senzory, které budou komunikovat prostřednictvím sběrnice I^2C, jež umožňuje připojit více zařízení na jednu komunikační linku.

Pro komunikaci s nadřazeným počítačem je zvolena sběrnice RS485, která je běžně používána v průmyslovém prostředí díky své vysoké odolnosti vůči rušení a schopnosti přenosu dat na větší vzdálenosti. Nad fyzickou vrstvou RS485 je implementován protokol Spinel, který umožňuje strukturovanou výměnu příkazů a dat mezi nadřazeným systémem a měřicím zařízením. Práce se zabývá jeho třemi vrstvami a podporou textového i binárního formátu zpráv [1].

Výpočetní část systému bude realizována pomocí mikrokontroléru, který poskytuje potřebná komunikační rozhraní, zejména sběrnici I^2C pro připojení teplotních senzorů a rozhraní pro komunikaci po sběrnici RS485. Architektura softwaru bude navržena tak, aby obsluha senzorů, zpracování protokolu Spinel i komunikace po sběrnici byly implementovány jako samostatné úlohy systému FreeRTOS. Oddělení jednotlivých funkcí do samostatných úloh zpřehlední strukturu programu a usnadní jeho další úpravy i rozšiřování. Zároveň umožní lépe řídit operace, které jsou citlivé na časování, například komunikaci a periodické měření.

Tato práce se zaměří především na návrh softwarové architektury pro zpracování protokolu Spinel tak, aby řešení bylo použitelné jak na straně mikrokontroléru, tak i na straně nadřazeného počítače. Součástí řešení bude návrh aplikační vrstvy umožňující připojení více teplotních senzorů na rozhraní I^2C, implementace komunikace přes RS485 a rozdělení systému do vzájemně nezávislých

komponent. Výsledkem práce bude funkční prototyp zařízení a jeho zhodnocení z hlediska stability, spolehlivosti a modularity.

Struktura práce je následující: druhá kapitola se zabývá popisem protokolu Spinel a jeho jednotlivých vrstev. Třetí kapitola je věnována návrhu softwaru komunikačního protokolu a čtvrtá kapitola popisuje jeho implementaci. V páté kapitole je popsán návrh a realizace měřicího modulu. Šestá kapitola se zaměřuje na implementaci aplikační vrstvy a firmware měřicího modulu. Sedmá kapitola obsahuje ověření funkce navrženého řešení a poslední kapitola shrnuje dosažené výsledky práce.

Kapitola 2

Protokol Spinel

Protokol Spinel je komunikační protokol vyvinutý společností Papouch s.r.o. [1] pro komunikaci mezi nadřazeným systémem a připojenými zařízeními. Jednou z jeho hlavních vlastností je vysoká modularita a možnost rozšiřování, která umožňuje přidávat nové funkce při zachování zpětné kompatibility mezi jednotlivými zařízeními.

Komunikace v protokolu Spinel probíhá ve formě rámců s definovaným začátkem a koncem zprávy. Přenos dat je založen na principu dotaz-odpověď, kdy nadřazený systém odesílá požadavek a zařízení následně vrací odpověď [1].

Protokol je rozdělen do tří vrstev, které zajišťují přenos rámců, strukturu přenášených dat a jejich interpretaci. Způsob kódování přenášených dat je určen formátem protokolu, přičemž jsou podporovány formáty textové i binární.

2.1 Architektura protokolu

Protokol Spinel lze z hlediska struktury rozdělit do tří logických vrstev. Toto rozdělení usnadňuje implementaci protokolu a odděluje jednotlivé části přenosu dat [1].

- **Frame** - zajišťuje přenos rámců mezi zařízeními a určuje jejich začátek, konec i formát.
- **SData** - určuje strukturu zprávy a nese informace pro identifikaci zařízení a příkazů.
- **Data** - obsahuje vlastní přenášená data zprávy.

Pod těmito logickými vrstvami se v navrženém systému nachází fyzická vrstva přenosu realizovaná prostřednictvím sběrnice RS485.

[1] `https://papouch.com`

2.2 Struktura rámce

Rámce protokolu Spinel mají definovanou strukturu, která se liší podle použitého formátu.

ASCII formát rámce je uveden v tabulce 2.1.

Tabulka 2.1: Struktura a popis ASCII rámce

Položka	**Popis**
PRE	Znak označující začátek zprávy
FRM	Identifikátor formátu zprávy
SDATA	Obsah zprávy
CR	Znak označující konec zprávy

Binární formát rámce rozšiřuje základní strukturu o dva bajty **NUM**, která společně určují délku přenášených dat. Jeho struktura je uvedena v tabulce 2.2.

Tabulka 2.2: Struktura a popis binárního rámce

Položka	**Popis**
PRE	Znak označující začátek zprávy
FRM	Identifikátor formátu
NUM	První bajt určující délku dat
NUM	Druhý bajt určující délku dat
SDATA	Obsah zprávy
CR	Znak označující konec zprávy

Konkrétní význam a struktura položky SDATA závisí na použitém formátu protokolu [1].

2.3 Formáty protokolu Spinel

Protokol Spinel podporuje několik komunikačních formátů, které se liší způsobem kódování přenášených dat. Formáty lze rozdělit na ASCII formáty, které používají textovou reprezentaci dat, a binární formáty, které umožňují efektivnější přenos většího množství dat.

V rámci této práce jsou relevantní především formáty 66 a 97, označované znaky `'B'`, respektive `'a'`, které byly využity při návrhu a implementaci komunikace. ASCII formát 66 byl využit zejména při ladění komunikace a ověřování správnosti přenášených dat, zatímco binární formát 97 byl použit pro standardní provoz zařízení. Formát 65 (`'A'`) je uveden pro úplnost, avšak v implementaci použit nebyl.

2.3.1 ASCII formát 65

Formát 65 představuje ASCII variantu protokolu Spinel s osmibitovou adresou a podpisem (SIG). Tento formát je vhodný zejména pro jednoduchou komunikaci a diagnostické účely, protože je snadno čitelný člověkem. Neobsahuje kontrolní součet a umožňuje testování komunikace například pomocí terminálových nástrojů.

Přenášená data jsou kódována pomocí ASCII znaků. Každý bajt je rozdělen na horní a dolní čtyřbitovou část, tzv. nibble. Tyto části jsou následně odesílány jako samostatné ASCII znaky odpovídající jejich hexadecimální hodnotě. Nejprve je odeslána horní část bajtu (MSN - Most Significant Nibble) a poté dolní část (LSN - Least Significant Nibble). Například bajt `D7h` je přenesen jako znaky `'D'` a `'7'` [1]. Struktura dotazu a odpovědi ve formátu 65 je uvedena v tabulkách 2.3 a 2.4. Význam základních položek v rámcích byl uveden v tabulkách 2.1 a 2.2.

Tabulka 2.3: Struktura dotazu ve formátu 65

PRE	FRM	ADR	SIG	INST	DATA	CR

Tabulka 2.4: Struktura odpovědi ve formátu 65

PRE	FRM	ADR	SIG	ACK	DATA	CR

2.3.2 ASCII formát 66

Formát 66 představuje ASCII variantu protokolu Spinel, která neobsahuje položku SIG ani kontrolní součet. Je určen především pro jednoduchá zařízení, jako jsou například senzory nebo měřicí moduly. Veškerá data jsou přenášena v textové podobě, což umožňuje snadnou čitelnost a jednoduché testování komunikace.

Na rozdíl od formátu 65 jsou data přenášena přímo jako ASCII znaky, bez převodu na hexadecimální reprezentaci po nibblech. Tento formát je proto vhodný zejména pro přenos jednoduchých textových nebo číselných hodnot [1]. Struktura dotazu a odpovědi ve formátu 66 je uvedena v tabulkách 2.5 a 2.6. Význam základních položek v rámcích je uveden v tabulkách 2.1 a 2.2.

Tabulka 2.5: Struktura dotazu ve formátu 66

PRE	FRM	ADR	INST	DATA	CR

Tabulka 2.6: Struktura odpovědi ve formátu 66

PRE	FRM	ADR	ACK	DATA	CR

2.3.3 Binární formát 97

Formát 97 je binární varianta protokolu Spinel s osmibitovou adresou, položkou SIG a kontrolním součtem. Je doporučen pro většinu nových zařízení díky efektivnějšímu přenosu dat oproti ASCII formátům. Data jsou přenášena přímo jako binární hodnoty, nikoli jako textové znaky. Rámec navíc obsahuje položku délky dat a kontrolní součet, který slouží k ověření integrity přijaté zprávy [1]. Struktura dotazu a odpovědi ve formátu 97 je uvedena v tabulkách 2.7 a 2.8. Význam základních položek v rámcích je uveden v tabulkách 2.1 a 2.2.

Tabulka 2.7: Struktura dotazu ve formátu 97

PRE	FRM	NUM	NUM	ADR	SIG	INST	DATA	SUMA	CR

Tabulka 2.8: Struktura odpovědi ve formátu 97

PRE	FRM	NUM	NUM	ADR	SIG	ACK	DATA	SUMA	CR

2.4 Význam jednotlivých položek rámce

Jednotlivé formáty protokolu Spinel využívají podobnou strukturu rámce, která je složena z několika základních položek. Tyto položky určují začátek a konec zprávy, identifikují použitý formát, adresu zařízení a obsah přenášených dat. V následující části jsou popsány jednotlivé položky používané ve zprávách protokolu Spinel [2].

2.4.1 PRE

Položka PRE (prefix) označuje začátek rámce a slouží k synchronizaci přijímajícího zařízení při zahájení příjmu zprávy. V protokolu Spinel je prefix reprezentován znakem `'*'`.

2.4.2 FRM

Položka FRM určuje použitý komunikační formát protokolu. Hodnota této položky definuje strukturu následujících dat a způsob jejich zpracování. Konkrétní formáty protokolu Spinel byly popsány v předchozí sekci.

2.4.3 NUM

Položka NUM určuje délku dat přenášených v binárním formátu 97. Jedná se o šestnáctibitovou hodnotu tvořenou dvěma po sobě jdoucími bajty, kde první bajt obsahuje vyšší část hodnoty a druhý bajt nižší část. Hodnota udává počet všech následujících bajtů až po ukončovací znak CR včetně.

Hodnota NUM zahrnuje SDATA, kontrolní součet, položky SUMA a CR. Minimální platná hodnota je 5, při nižší hodnotě je zpráva považována za chybnou.

2.4.4 ADR

Položka ADR určuje adresu zařízení na komunikační lince. Zařízení tuto adresu uvádí ve svých odpovědích nadřazenému systému.

Ve formátech 65 a 97 je adresa reprezentována jedním bajtem. Ve formátu 65 je tento bajt při přenosu rozdělen na dvě čtyřbitové části (nibbly) a odesílán jako dvojice ASCII znaků odpovídajících hexadecimálním hodnotám. V binárním formátu 97 je adresa přenášena přímo jako jeden bajt.

Ve formátu 66 je adresa reprezentována jedním ASCII znakem. Povolené jsou znaky číslic `0-9`, velkých písmen `A-Z` a malých písmen `a-z`. Adresa nesmí být shodná s prefixem ani se znakem ukončujícím rámec.

Protokol Spinel definuje také speciální adresy určené pro komunikaci s více zařízeními na jedné lince. V binárním formátu 97 a ve formátu 65 je hodnota `FFh` použita jako broadcast adresa, na kterou zařízení neodpovídají. Hodnota `FEh` představuje univerzální adresu, která se používá v případě, že je na lince připojeno pouze jedno zařízení. Ve formátu 66 je broadcast reprezentován znakem `'%'` a univerzální adresa znakem `'$'`.

2.4.5 SIG

Položka SIG slouží k jednoznačné identifikaci konkrétního dotazu. Hodnota této položky je zvolena při odeslání dotazu a stejná hodnota je následně uvedena i v odpovědi zařízení. Díky tomu lze jednoduše určit, na který dotaz přijatá odpověď patří.

Položka SIG je použito ve formátech 65 a 97. Ve formátu 66 se tato položka nevyskytuje.

2.4.6 INST

Položka INST obsahuje kód instrukce, která určuje požadovanou operaci zařízení. Na základě hodnoty této položky zařízení vyhodnotí přijatý dotaz a provede odpovídající operaci. Pokud zařízení přijatý kód instrukce nezná, vrací v odpovědi chybový kód ACK označující neplatnou instrukci.

2.4.7 ACK

Položka ACK informuje nadřazený systém o výsledku zpracování přijaté instrukce. V odpovědi zařízení nahrazuje položku INST a obsahuje kód, který určuje, zda byla instrukce provedena úspěšně nebo zda během zpracování došlo k chybě. Přehled definovaných kódů je uveden v tabulce 2.9, která vychází z implementační dokumentace protokolu Spinel [2].

Tabulka 2.9: Kódy potvrzení ACK

Formát 97/65	**Formát 66**	**Význam**
00h	0	Vše v pořádku
01h	1	Jiná chyba
02h	2	Neplatný kód instrukce
03h	3	Neplatná data
04h	4	Nepovolen zápis / přístup odmítnut
05h	5	Porucha zařízení
0Ch	C	Automatická instrukce – přenos dat
0Dh	D	Automatická instrukce – změna logické hodnoty
0Eh	E	Automatická instrukce – kontinuální měření
0Fh	F	Automatická instrukce – ostatní

2.4.8 DATA

Položka DATA obsahuje vlastní data přenášená v rámci instrukce nebo odpovědi. Jeho význam a délka závisejí na konkrétní použité instrukci.

Pokud délka nebo obsah dat neodpovídají požadovanému formátu, zařízení instrukci nezpracuje a vrátí odpověď s chybovým kódem ACK označujícím neplatná data.

2.4.9 SUMA

Položka SUMA představuje kontrolní součet používaný pro ověření integrity přenášené zprávy v binárním formátu 97. Hodnota kontrolního součtu je vypočtena jako doplněk součtu všech přenášených bajtů kromě ukončovacího znaku CR do hodnoty 255 [2].

$$SUMA = 255 - (PRE + FRM + NUM + SDATA) \tag{2.1}$$

Pokud přijatá zpráva obsahuje neplatný kontrolní součet, zařízení na ni neodpovídá.

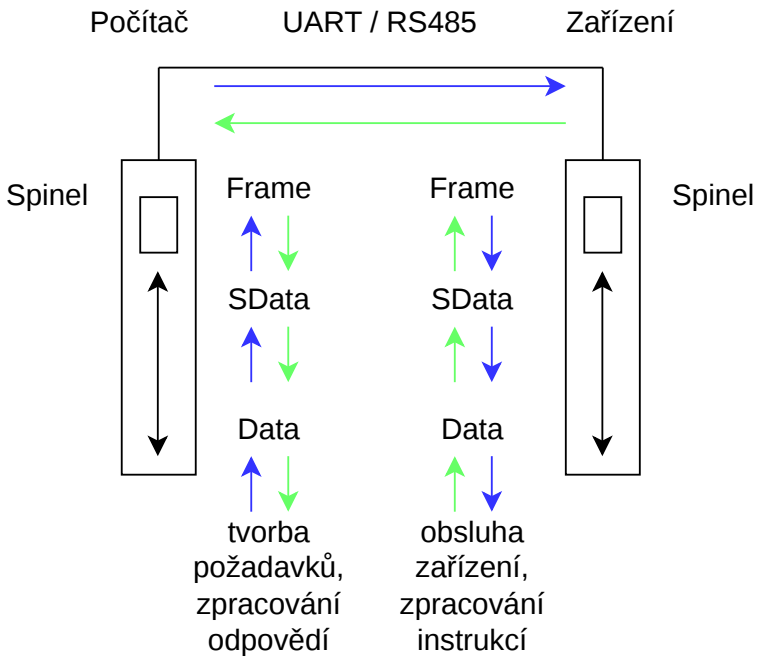
2.4.10 CR

Položka CR označuje konec přenášeného rámce. Přijímající zařízení podle tohoto znaku rozpozná, že byl přenos rámce dokončen. V protokolu Spinel je ukončení rámce reprezentováno hodnotou `0Dh`.

Kapitola 3

Návrh softwaru komunikačního protokolu

Softwarová implementace komunikačního protokolu byla navržena jako sada sdílených modulů použitelných jak na straně mikrokontroléru, tak na straně nadřazeného počítače. Cílem návrhu bylo oddělit jednotlivé odpovědnosti do samostatných vrstev tak, aby bylo možné protokol jednoduše rozšiřovat, testovat a přenášet mezi platformami. Architektura je založena na vrstevném zpracování dat, kde každá vrstva odpovídá za přesně vymezenou část komunikačního procesu. Přehled této architektury je znázorněn na obrázku 3.1.



Obrázek 3.1: Vrstvené uspořádání softwarové implementace protokolu Spinel pro nadřazený počítač a mikrokontrolér

3.1 Návrh protokolových vrstev

Navržené zpracování protokolu je rozděleno do několika logických vrstev. Nejnižší vrstvu tvoří transportní rozhraní UART, které zajišťuje fyzický přenos dat mezi zařízeními. Nad touto vrstvou je umístěna vrstva `Frame` (viz sekci 2.1), která odpovídá za tvorbu a rozkládání fyzického rámce protokolu, včetně synchronizačního znaku PRE, identifikátoru formátu rámce, délkového pole, kontrolního součtu a ukončovacího znaku CR.

Další vrstvu představuje `SData` (viz sekci 2.1), která pracuje s obsahem rámce a interpretuje jednotlivé části protokolu, zejména adresu zařízení, podpis a přenášený obsah. Instrukce nebo potvrzovací kód zůstává součástí předávaného datového obsahu pro další aplikační interpretaci.

Nejvyšší část zpracování představuje aplikační vrstva, která interpretuje význam přenášených dat a zajišťuje vlastní obsluhu požadavků. Její konkrétní chování se může lišit podle strany komunikace i podle účelu zařízení, zatímco nižší vrstvy zůstávají společné.

3.2 Správce protokolu

Hlavním prvkem navržené architektury je třída `SpinelManager`, která propojuje jednotlivé vrstvy protokolu a řídí tok dat mezi přijímací frontou, jednotlivými vrstvami protokolu a aplikační částí systému. Její hlavní úlohou je převzít přijatý rámec, postupně jej rozložit pomocí `Frame` a `SData` a získaný datový obsah předat aplikační vrstvě ke zpracování.

Po zpracování přijatého požadavku zajišťuje tato třída také sestavení odpovědi, její opětovné složení do odpovídajícího formátu rámce a odeslání prostřednictvím rozhraní UART. Zpracování využívá abstrakci přijímací fronty, jež odděluje příjem dat od jejich následné interpretace. Díky tomu může být příjem rámců realizován nezávisle na konkrétní hardwarové platformě.

3.3 Zpracování příchozího rámce

Příjem dat probíhá po jednotlivých bajtech, takže přijatou zprávu nelze bez předchozího vyhodnocení přímo předat vrstvě `Frame`. Je nejprve nutné rozpoznat začátek a konec rámce, určit, zda se jedná o ASCII nebo binární variantu rámce, a současně zachytit chybové stavy, které mohou během příjmu vzniknout.

Parser je navržen jako samostatná část zajišťující předzpracování přijatých dat. Jeho úlohou je rozlišit jednotlivé varianty rámce, určit hranice přijímaného rámce a připravit jej pro další zpracování ve vrstvě `Frame`. Současně musí evidovat chybové stavy vzniklé během příjmu, aby mohly být zohledněny při dalším zpracování rámce.

Po dokončení příjmu je rámec uložen do přijímací fronty, odkud jej následně přebírá správce protokolu k dalšímu zpracování.

3.4 Vrstva Frame

`Frame` zajišťuje zpracování fyzické podoby přenášeného rámce. Při příjmu odděluje vlastní obsah zprávy od režijních částí rámce, jako jsou synchronizační znak, identifikátor formátu, délkové pole, kontrolní součet a ukončovací znak. Při odesílání naopak tyto části k datovému obsahu doplňuje a vytváří tak kompletní rámec připravený k přenosu.

Vrstva `Frame` musí současně zohlednit, že protokol podporuje různé varianty rámce. Podle použitého formátu se liší obsah režijních částí i způsob vymezení vlastního obsahu zprávy, který je následně předáván vrstvě `SData`.

3.5 Vrstva SData

`SData` navazuje na `Frame` a pracuje s vnitřním obsahem zprávy bez režijních částí fyzického rámce. Jejím úkolem je interpretovat přijatý obsah, oddělit z něj adresaci a podle použitého formátu případně také podpis a připravit datový obsah pro aplikační zpracování.

Při příjmu z přenášeného obsahu odděluje adresu zařízení a podle použitého formátu případně také podpis. Zbylá část zprávy je následně předána vyšší vrstvě jako datový obsah určený pro další interpretaci. Instrukce nebo potvrzovací kód přitom zůstává součástí tohoto obsahu.

Při odesílání postupuje opačným směrem. K datovému obsahu připravenému aplikační vrstvou doplňuje adresu zařízení a podle zvoleného formátu případně také podpis. Výsledkem je obsah zprávy připravený k předání vrstvě `Frame`.

3.6 Shrnutí návrhu

Navržená softwarová architektura vychází z obecného popisu protokolu Spinel. Pro potřeby implementace byly v rámci této práce jednotlivé části protokolu uspořádány do tří logických vrstev, které oddělují fyzický přenos, strukturu zprávy a aplikační interpretaci dat. Toto rozdělení umožňuje sdílet základní části implementace mezi mikrokontrolérem a nadřazeným počítačem. Tím vzniká základ pro následnou implementaci komunikačního protokolu i aplikační logiky.

Kapitola 4

Implementace komunikačního protokolu

Na základě navržené architektury byla vytvořena implementace komunikačního protokolu Spinel, sdílená mezi mikrokontrolérem a nadřazeným počítačem. Cílem bylo zachovat společné jádro protokolu nezávislé na konkrétní platformě a oddělit od něj pouze části závislé na hardwaru.

Implementace vychází z rozdělení protokolu do vrstev popsaných v předchozí kapitole. Jednotlivé části protokolu byly realizovány jako samostatné moduly odpovídající modulu `Parser`, vrstvě `Frame`, vrstvě `SData` a správci protokolu. Tyto moduly doplňuje aplikační vrstva, která na straně mikrokontroléru zajišťuje interpretaci přijatých instrukcí a sestavení odpovědi, zatímco na straně počítače slouží k vytváření požadavků a zpracování odpovědí.

4.1 Implementace parseru

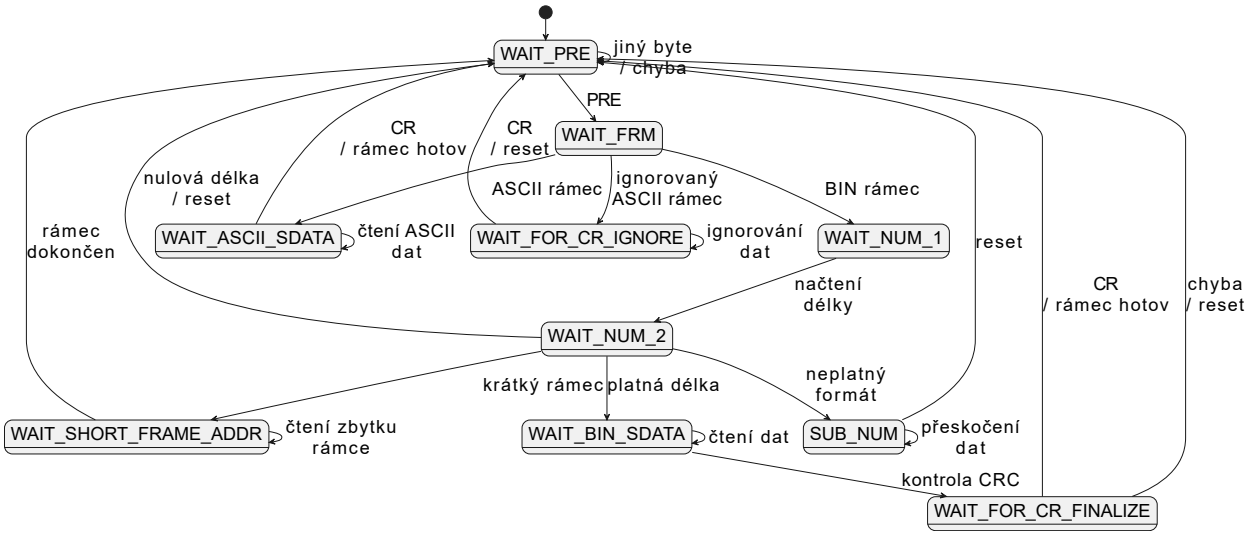
Parser dat je implementován jako stavový automat, který zpracovává vstup po jednotlivých bajtech. Po přijetí synchronizačního znaku určuje variantu rámce a podle ní nastavuje další postup příjmu. U binárního formátu sleduje údaj o délce rámce a očekávaný počet bajtů, zatímco u ASCII varianty vyhodnocuje ukončení rámce pomocí ukončovacího znaku.

Během příjmu parser průběžně eviduje chybové stavy, například přetečení bufferu, neúplný rámec nebo chybný kontrolní součet. Tyto informace jsou ukládány do příznaků struktury `RxBuffer`, která slouží jako výstup parseru. Po dokončení příjmu je sestavený rámec uložen do přijímací fronty, odkud jej přebírá správce protokolu.

Stavy parseru a přechody mezi nimi jsou znázorněny na obrázku 4.1.

Výchozí stav `WAIT_PRE` představuje čekání na synchronizační znak začátku rámce. Ve stavu `WAIT_FRM` parser vyhodnocuje identifikátor formátu a rozhoduje o dalším způsobu zpracování rámce. Stavy `WAIT_NUM_1` a `WAIT_NUM_2` slouží ke zpracování délkového pole binárního rámce.

Pokud je délka rámce platná, parser pokračuje do stavu `WAIT_BIN_SDATA`, ve kterém přijímá datovou část rámce a současně připravuje kontrolu kontrolního součtu. Stav `WAIT_FOR_CR_FINALIZE`



Obrázek 4.1: Stavový automat parseru protokolu Spinel

zajišťuje dokončení příjmu binárního rámce a ověření jeho korektního ukončení. Stav `WAIT_ASCII_SDATA` slouží k příjmu dat ASCII varianty rámce až do přijetí ukončovacího znaku.

Pomocné stavy `WAIT_FOR_CR_IGNORE`, `WAIT_SHORT_FRAME_ADDR` a `SUB_NUM` slouží ke zpracování nepodporovaných nebo chybných rámců, případně k jejich korektnímu ukončení před návratem do výchozího stavu.

Výstupem parseru je struktura `RxBuffer`, která uchovává přijatá data, délku rámce a příznaky chybových stavů. Její podoba je uvedena ve výpisu 4.1.

```
struct RxBuffer{
    static constexpr size_t CAPACITY = 256;
    std::array<uint8_t, CAPACITY> data{};
    size_t len = 0;
    uint8_t flags = 0;

    void append(uint8_t byte){
        if(len >= CAPACITY){
            flags |= RxFlags::OVERFLOW;
            return;
        }
        data[len++] = byte;
    }

    void clear(){
```

```
        len = 0;
        flags = 0;
    }

    SpinelBufferView getView() const {
        return { data.data(), len };
    }

    uint8_t operator[](size_t index) const {
        return data[index];
    }
};
```

Listing 4.1: Struktura RxBuffer

4.2 Implementace vrstvy Frame

`Frame` zajišťuje převod mezi fyzickou podobou přenášené zprávy a jejím vnitřním obsahem určeným pro další zpracování. Na straně příjmu z přijatého rámce odděluje režijní části protokolu a vytváří strukturu `UnwrappedFrame`, která obsahuje vlastní obsah zprávy a informaci o použité variantě rámce. Její podoba je uvedena ve výpisu 4.2.

```
struct UnwrappedFrame{
    SpinelBufferView payload;
    bool isAscii = false;
    bool frameValid = false;
};
```

Listing 4.2: Struktura UnwrappedFrame

Při příjmu nejprve určuje, zda se jedná o ASCII nebo binární variantu rámce. U binárního formátu odděluje od vlastního obsahu zprávy úvodní část rámce obsahující synchronizační znak, identifikátor formátu a délkové pole i závěrečnou část tvořenou kontrolním součtem a ukončovacím znakem. U ASCII varianty je vyšší vrstvě předán obsah mezi identifikátorem formátu a ukončovacím znakem.

Při odesílání postupuje opačným směrem. K datovému obsahu připravenému vyšší vrstvou doplňuje synchronizační znak, identifikátor formátu a podle zvoleného typu rámce také další potřebné

položky, konkrétně u binárního formátu délku rámce a vypočítaný kontrolní součet. Výsledkem je úplný rámec uložený do výstupního bufferu, připravený k odeslání po komunikační lince.

4.3 Implementace vrstvy SData

`SData` navazuje na `Frame` a pracuje s vnitřním obsahem zprávy bez režijních částí fyzického rámce. Jejím výstupem je struktura `UnwrappedSData`, která obsahuje adresaci, případně podpis, a datový obsah určený pro aplikační zpracování. Její podoba je uvedena ve výpisu 4.3.

```
struct UnwrappedSData{
    SpinelBufferView payload;
    uint8_t addr;
    uint8_t sig = 0;
    bool addrValid = false;
    bool sigValid = false;
};
```

Listing 4.3: Struktura UnwrappedSData

Při příjmu z přenášeného obsahu odděluje adresu zařízení a podle použitého formátu případně také podpis. Zbylá část zprávy je následně předána vyšší vrstvě jako datový obsah určený pro další interpretaci. Instrukce nebo potvrzovací kód přitom zůstává součástí tohoto obsahu.

Při odesílání postupuje opačným směrem. K datovému obsahu připravenému aplikační vrstvou doplňuje adresu zařízení a podle zvoleného formátu případně také podpis. Pokud použitý formát podpis obsahuje, je v odpovědi použit stejný podpis, jaký byl přijat v původním rámci. Výsledkem je strukturovaný obsah zprávy připravený k předání vrstvě `Frame`.

4.4 Implementace správce protokolu

Správce protokolu představuje řídicí prvek celé implementace, který propojuje vrstvy `Frame`, `SData` a aplikační vrstvu prostřednictvím abstraktních rozhraní `IFrameQueue`, `IUartLayer` a `ISpinelApplication`. Jeho úlohou je postupně předat přijatý rámec jednotlivým vrstvám ke zpracování a podle výsledku zajistit vytvoření a odeslání odpovědi.

Rozhraní mezi správcem protokolu a aplikační vrstvou je realizováno pomocí abstraktní třídy `ISpinelApplication`, jejíž podoba je uvedena ve výpisu 4.4.

```
class ISpinelApplication{
```

```
public:
    virtual ~ISpinelApplication() = default;
    virtual TxBuffer onFrameReceived(const RxBuffer& rawRx, const UnwrappedSData&
        sdata, bool isAscii) = 0;
    virtual void onResponseSent(bool needReset) = 0;
};
```

Listing 4.4: Rozhraní ISpinelApplication

Po převzetí rámce správce nejprve využije `Frame` k oddělení vlastního obsahu zprávy od režijních částí přenosu. Poté předá získaný obsah vrstvě `SData`, která z něj oddělí adresaci a podle použitého formátu případně také podpis. Takto připravený datový obsah je předán aplikační vrstvě.

Pokud aplikační vrstva vytvoří odpověď, správce protokolu zajistí její zpětné sestavení opačným směrem přes `SData` a `Frame`. Výsledný rámec je následně odeslán prostřednictvím rozhraní UART. Správce protokolu dále zajišťuje změnu komunikační rychlosti, pokud ji vyžaduje zpracovaná instrukce.

4.5 Shrnutí implementace

Implementace komunikačního protokolu vychází z návrhu rozděleného do samostatných vrstev a zachovává společné jádro použitelné na obou stranách komunikace. Jednotlivé vrstvy byly realizovány jako samostatné moduly, které zajišťují příjem rámce, jeho rozbalení, interpretaci a následné sestavení odpovědi. Díky využití společných datových struktur a abstraktních rozhraní bylo dosaženo modulární a přenositelné implementace nezávislé na konkrétní hardwarové platformě.

Kapitola 5

Návrh a realizace měřicího modulu

Návrh měřicího modulu navazuje na předchozí kapitoly věnované protokolu Spinel a jeho softwarové implementaci. Cílem této kapitoly je popsat hardwarovou platformu, na které je navržený systém realizován, a způsob propojení jednotlivých komponent. Součástí kapitoly je také zdůvodnění volby použitých hardwarových prvků s ohledem na požadavky zadání.

Při návrhu bylo nutné zohlednit požadavky zadání a zvolit vhodnou hardwarovou platformu pro realizaci navrženého systému. Po konzultaci s vedoucím práce proto byla jako náhrada za vývojovou platformu FRDM-MCXA153 uvedenou v zadání zvolena platforma FRDM-MCXN947. Ta je založena na mikrokontroléru ze stejné rodiny MCX a využívá nejnovější jádro ARM Cortex-M33, takže svými vlastnostmi vyhovuje požadavkům navrženého řešení. Současně byla snadno dostupná, protože je používána ve výuce předmětů APPS a OSY.

5.1 Požadavky na měřicí modul

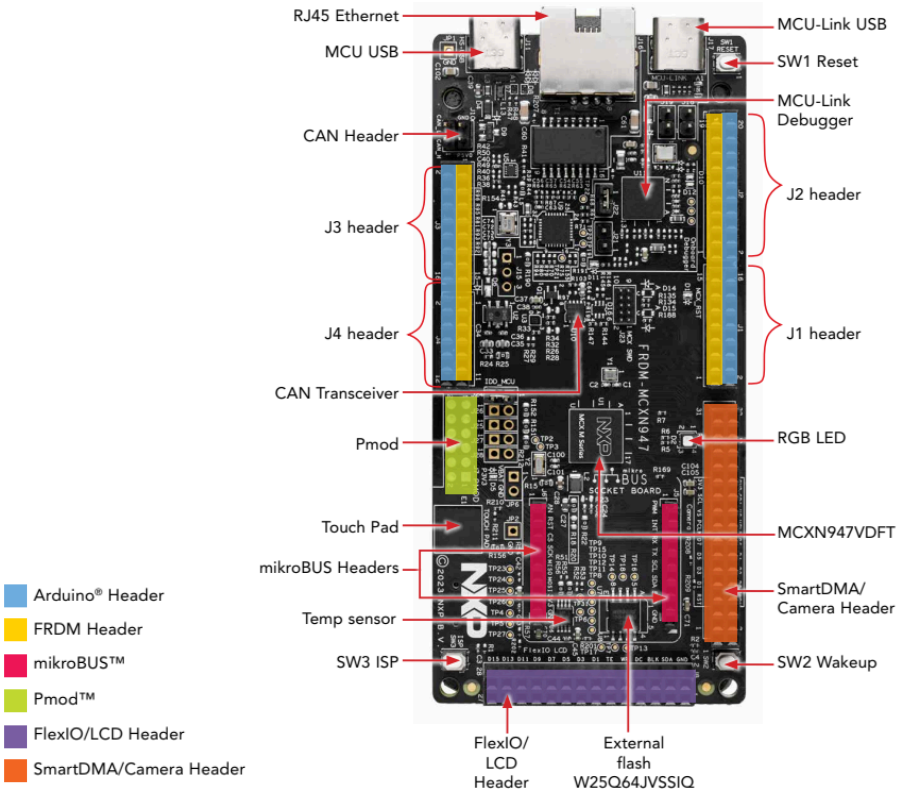
Měřicí modul byl navržen jako zařízení určené pro připojení k nadřazenému počítači prostřednictvím sběrnice RS485 a pro měření teploty na více místech pomocí senzorů TMP275. Návrh musel splnit požadavek na použití mikrokontroléru s jádrem ARM a podporou operačního systému FreeRTOS, přičemž bylo nutné umožnit připojení více senzorů prostřednictvím sběrnice I^2C.

Důležitým požadavkem byla také modularita řešení, která umožňuje oddělit komunikační část od části určené pro měření a usnadňuje případné rozšíření systému o další senzory. Návrh proto vychází z použití vývojové desky FRDM-MCXN947, externího převodníku pro komunikaci po sběrnici RS485 a teplotních senzorů připojených přes sběrnici I^2C.

Z hlediska funkce bylo dále nutné zajistit spolehlivou komunikaci s nadřazeným systémem, periodické měření teploty a možnost zpracování požadavků přijatých prostřednictvím protokolu Spinel. Použitá hardwarová platforma proto musí poskytovat dostatečný výpočetní výkon, rozhraní UART pro komunikaci prostřednictvím externího převodníku po sběrnici RS485, sběrnici I^2C pro připojení teplotních senzorů a současně podporu pro implementaci více úloh v prostředí FreeRTOS.

5.2 Vývojová deska FRDM-MCXN947

Jako základ řídicí části měřicího modulu byla zvolena vývojová deska FRDM-MCXN947 společnosti NXP Semiconductors[1] [3]. Deska je osazena mikrokontrolérem MCXN947 z rodiny MCX N a svými parametry vyhovuje požadavkům navrženého řešení. Poskytuje potřebná komunikační rozhraní a současně umožňuje snadný vývoj i ověřování výsledného programu. Použitá vývojová deska je zobrazena na obrázku 5.1.



Obrázek 5.1: Vývojová deska FRDM-MCXN947 s vyznačenými hlavními částmi a typy konektorů. Převzato a upraveno podle [3]

Mikrokontrolér MCXN947 je vybaven dvěma procesorovými jádry ARM Cortex-M33 s maximální taktovací frekvencí 150 MHz [4]. K dispozici je interní programová paměť Flash o kapacitě 2 MB a operační paměť SRAM o kapacitě 512 kB [4]. Tyto parametry poskytují dostatečnou rezervu pro implementaci aplikační logiky, komunikačních funkcí i využití operačního systému reálného času.

[1] `https://www.nxp.com`

Současně umožňují uchovávat programová data, přijímací a vysílací buffery komunikačních rozhraní i naměřené hodnoty ze senzorů bez výraznějších paměťových omezení.

Z hlediska navrženého zařízení jsou důležitá především integrovaná komunikační rozhraní mikrokontroléru. MCXN947 obsahuje více periferních bloků Flexcomm, které lze podle potřeby konfigurovat například jako UART, I^2C nebo SPI [4]. To umožňuje využít na jedné platformě více komunikačních rozhraní současně.

V navrženém řešení je rozhraní UART použito pro komunikaci s nadřazeným počítačem přes sběrnici RS485 s využitím převodníku MAX485, zatímco sběrnice I^2C slouží pro připojení teplotních senzorů TMP275. Využití samostatných periferií pro obě komunikační větve zjednodušuje návrh programu, protože komunikace s nadřazeným systémem a obsluha senzorů mohou být řešeny odděleně.

Pro správnou funkci komunikačních periferií i časově deterministického chování aplikace je důležitá také soustava hodinových zdrojů mikrokontroléru. K dispozici jsou interní oscilátory FRO 12 MHz, FRO 144 MHz a FRO 16 kHz, dále nízkopříkonový oscilátor 32 kHz, externí krystalový oscilátor až do 50 MHz a dvojice PLL obvodů [4]. Tato výbava umožňuje nastavit systémové i periferní hodinové signály podle požadavků aplikace, zejména s ohledem na správné časování sériové komunikace a periodické vyhodnocování dat ze senzorů.

Vývojová deska FRDM-MCXN947 byla zvolena také z praktických důvodů. Umožňuje snadné prototypování, poskytuje hardwarovou podporu pro ladění a je podporována vývojovým prostředím MCUXpresso [3]. To usnadňuje konfiguraci periferií, tvorbu projektu i následné ladění programu. Pro implementaci řídicí logiky je navíc možné využít operační systém FreeRTOS, který umožňuje rozdělit aplikaci do samostatných úloh. V navrženém systému tak lze oddělit například úlohu zajišťující komunikaci po sběrnici RS485 od úlohy periodického čtení teplotních senzorů a zpracování naměřených dat.

Z uvedených důvodů představuje vývojová deska FRDM-MCXN947 vhodnou hardwarovou platformu pro realizaci navrženého měřicího zařízení. Nabízí potřebný výpočetní výkon, komunikační rozhraní i paměťovou kapacitu a současně usnadňuje vývoj a ladění výsledného programu.

5.3 Převodník MAX485 a komunikace po sběrnici RS485

Komunikace mezi měřicím modulem a nadřazeným počítačem je realizována prostřednictvím sběrnice RS485. Nadřazený počítač je ke sběrnici připojen přes externí převodník USB/RS485, zatímco na straně měřicího modulu je použit převodník MAX485. Ten umožňuje převod mezi rozhraním UART mikrokontroléru a sběrnicí RS485.

Obvod MAX485 je transceiver určený pro komunikaci podle standardu RS485. Obsahuje vysílací i přijímací část a zajišťuje převod mezi nesymetrickými úrovněmi rozhraní UART a diferenciálními signály sběrnice RS485 [5]. V navrženém zapojení je proto převodník MAX485 zařazen mezi rozhraní UART mikrokontroléru a komunikační vodiče sběrnice.

Použití sběrnice RS485 je výhodné zejména z hlediska způsobu přenosu dat po diferenciální dvojici vodičů. Tento princip umožňuje spolehlivou komunikaci i na větší vzdálenosti než při použití běžného nesymetrického rozhraní UART. V navrženém systému však byla komunikace realizována na krátkou vzdálenost v laboratorním zapojení.

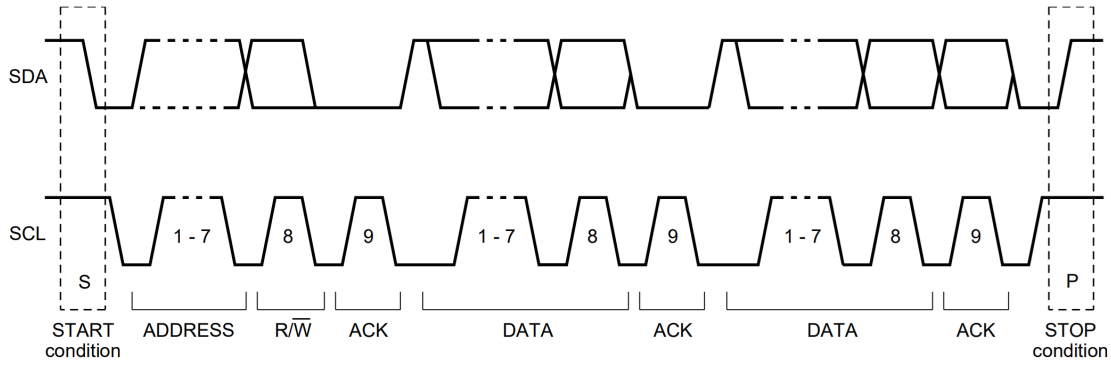
MAX485 je určen pro poloduplexní komunikaci, což pro navržený měřicí modul postačuje [5]. Z hlediska návrhu je výhodné, že použití externího transceiveru odděluje fyzickou vrstvu komunikace od vlastního řízení protokolu. Mikrokontrolér tak může využívat standardní periférii UART bez nutnosti přímé podpory rozhraní RS485, zatímco převodník MAX485 zajišťuje elektrické přizpůsobení signálů pro přenos po sběrnici RS485.

5.4 Sběrnice I²C a připojení senzorů

Pro připojení teplotních senzorů k mikrokontroléru byla v navrženém zařízení zvolena sběrnice I²C. Jedná se o synchronní sériovou sběrnici, která využívá dvě signálové linky, datovou linku SDA a hodinovou linku SCL [6]. Výhodou sběrnice I²C je, že jde o běžně používané rozhraní pro připojení periferních obvodů k mikrokontroléru. V navrženém zařízení je vhodná zejména proto, že umožňuje připojit více senzorů na jednu společnou sběrnici.

Komunikace na sběrnici I²C probíhá mezi řídicím zařízením a připojenými podřízenými zařízeními. V navrženém měřicím modulu vystupuje mikrokontrolér jako řídicí zařízení, zatímco jednotlivé senzory TMP275 tvoří podřízená zařízení. Každý senzor je na sběrnici rozlišen vlastní adresou, což umožňuje připojit více senzorů na jednu společnou sběrnici [6].

Přenos dat po sběrnici I²C probíhá sériově po jednotlivých bitech synchronizovaných hodinovým signálem SCL. Komunikace je zahájena podmínkou START a ukončena podmínkou STOP. Na začátku přenosu je vyslána adresa zařízení a bit určující směr komunikace, po nichž následuje potvrzovací bit ACK. Další data jsou přenášena po jednotlivých bajtech, přičemž po každém bajtu následuje potvrzovací bit ACK [6]. Příklad úplného průběhu přenosu dat na sběrnici I²C je znázorněn na obrázku 5.2.



Obrázek 5.2: Příklad úplného přenosu dat po sběrnici I²C. Převzato z [6]

V navrženém systému je sběrnice I^2C využita pro komunikaci s teplotními senzory TMP275. Mikrokontrolér prostřednictvím této sběrnice adresuje jednotlivé senzory a zajišťuje čtení naměřených hodnot. Díky možnosti připojit více senzorů na jednu sběrnici představuje I^2C vhodné řešení pro propojení mikrokontroléru s několika teplotními senzory současně.

Z praktického hlediska je při použití sběrnice I^2C nutné správně zapojit signály SDA a SCL a zajistit jejich klidový stav sběrnice pomocí pull-up rezistorů.

5.5 Teplotní senzory TMP275

Pro měření teploty v navrženém zařízení byl zvolen digitální teplotní senzor TMP275 společnosti Texas Instruments [7]. Tento senzor komunikuje prostřednictvím sběrnice I^2C, a je proto vhodný pro připojení k mikrokontroléru použitému v navrženém měřicím modulu. Jeho výhodou je zejména jednoduché digitální rozhraní a parametry vyhovující požadavkům navržené aplikace.

Senzor TMP275 umožňuje měření teploty v rozsahu -40 °C do 125 °C [7]. Pracuje s napájecím napětím v rozsahu 2.7 až 5.5 V, což umožňuje jeho snadné použití ve vestavných systémech. Naměřená teplota je ukládána do teplotního registru, z něhož ji mikrokontrolér čte prostřednictvím sběrnice I^2C.

Výhodou senzoru TMP275 je možnost volby adresy na sběrnici I^2C pomocí adresových vstupů `A0`, `A1` a `A2` [7]. Adresa zařízení je tvořena pevně danou částí a třemi bity nastavenými právě těmito vstupy. Tyto vstupy se připojují na logickou úroveň odpovídající `GND` nebo `V+`, čímž lze zvolit jednu z osmi adres zařízení. Díky tomu je možné na jednu společnou sběrnici I^2C připojit více senzorů TMP275 současně.

Naměřená teplota je dostupná v teplotním registru senzoru, který je určen pouze pro čtení [7]. Hodnota teploty je přenášena ve dvou bajtech, přičemž nejprve je vysílán bajt s vyšší platností. Mikrokontrolér v navrženém systému z tohoto registru získává aktuální hodnoty teplot jednotlivých senzorů.

Senzor dále obsahuje ukazatel registru, pomocí kterého se volí registr určený pro další komunikaci, a také konfigurační registr, který umožňuje nastavení provozních vlastností senzoru [7]. V navrženém zařízení je důležité zejména nastavení rozlišení převodu teploty. TMP275 umožňuje volit rozlišení v rozsahu 9 až 12 bitů, čemuž odpovídá teplotní krok 0.5 °C, 0.25 °C, 0.125 °C a 0.0625 °C [7]. S rostoucím rozlišením se současně prodlužuje doba konverze.

Kromě samotného měření teploty obsahuje TMP275 také funkci termostatu s výstupem ALERT a registry pro nastavení mezních teplot [7]. Tato funkce však v navrženém zařízení nebyla využita, protože senzor slouží pouze pro periodické měření teploty. Konfigurační registr je proto využíván především pro nastavení základního režimu činnosti senzoru.

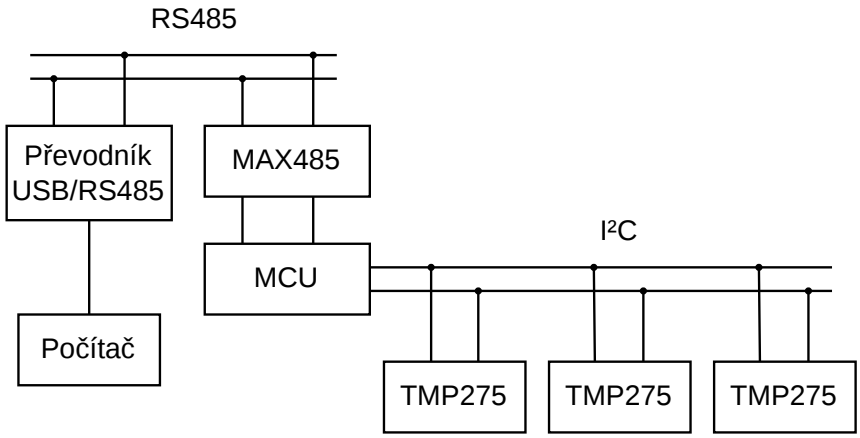
Z uvedených důvodů představuje TMP275 vhodný teplotní senzor pro navržený měřicí modul. Poskytuje parametry vyhovující požadavkům navrženého systému a současně umožňuje jednoduché digitální připojení k mikrokontroléru.

5.6 Celkové hardwarové zapojení

Celkové hardwarové zapojení navrženého systému vychází z propojení vývojové desky FRDM-MCXN947, převodníku MAX485 a teplotních senzorů TMP275. Mikrokontrolér tvoří řídicí část celého zařízení, zajišťuje komunikaci s nadřazeným počítačem prostřednictvím převodníku MAX485 a sběrnice RS485 a současně obsluhuje teplotní senzory připojené přes sběrnici I^2C.

Na straně komunikace s nadřazeným systémem je využito rozhraní UART mikrokontroléru, které je prostřednictvím převodníku MAX485 připojeno ke sběrnici RS485. Na straně měření teploty jsou senzory TMP275 připojeny ke sběrnici I^2C pomocí signálů SDA a SCL. Díky adresaci jednotlivých senzorů je možné připojit více senzorů na jednu společnou sběrnici.

Takto navržené zapojení odděluje komunikační část systému od části určené pro měření teploty a vytváří hardwarový základ pro implementaci protokolu Spinel i aplikační logiky měřicího modulu. Blokové schéma navrženého systému je znázorněno na obrázku 5.3.

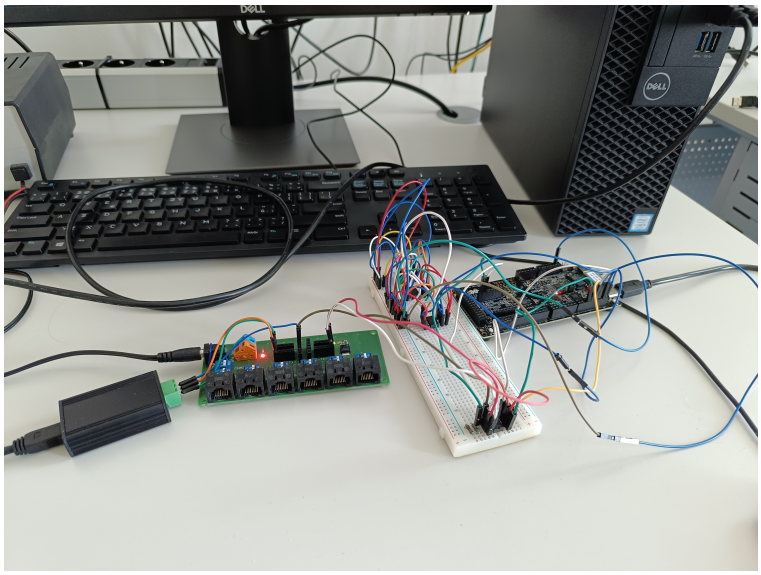


Obrázek 5.3: Blokové schéma hardwarového propojení nadřazeného počítače, měřicího modulu a teplotních senzorů TMP275

Praktická realizace navrženého zapojení v laboratorních podmínkách je znázorněna na obrázku 5.4.

5.7 Shrnutí kapitoly

V této kapitole byly popsány hlavní hardwarové části navrženého měřicího modulu a jejich vzájemné propojení. Jako základ systému byla zvolena vývojová deska FRDM-MCXN947 s mikrokontrolérem MCXN947, která poskytuje potřebný výpočetní výkon, komunikační rozhraní i podporu pro vývoj řídicího programu. Pro komunikaci s nadřazeným počítačem byla použita sběrnice RS485 realizovaná



Obrázek 5.4: Laboratorní zapojení měřicího modulu použité při ověřování komunikace po sběrnici RS485 a měření teploty pomocí senzorů TMP275

pomocí převodníku MAX485. Měření teploty je zajištěno senzory TMP275 připojenými přes sběrnici I^2C.

Navržené hardwarové řešení splňuje požadavky zadání a současně vytváří základ pro následnou implementaci aplikační logiky, komunikace se senzory i běhu systému v prostředí FreeRTOS.

Kapitola 6

Implementace aplikační vrstvy a firmware měřicího modulu

Kapitola navazuje na předchozí části práce věnované návrhu komunikačního protokolu a hardwarové realizaci měřicího modulu. Jejím cílem je popsat programové vybavení mikrokontroléru, způsob rozdělení programu do jednotlivých částí a princip zpracování komunikace, měření teploty a ukládání konfiguračních údajů.

Firmware měřicího modulu byl navržen jako modulární aplikace běžící pod operačním systémem FreeRTOS. Jednotlivé části programu jsou odděleny podle své funkce, aby bylo možné přehledně řešit obsluhu komunikační linky, periodické čtení teplotních senzorů, zpracování aplikačních příkazů i ukládání konfigurace do interní paměti Flash. Takové rozdělení usnadňuje další rozšiřování programu a zpřehledňuje jeho strukturu. Z pohledu běhu programu je firmware rozdělen do několika úloh operačního systému FreeRTOS. Samostatná úloha zajišťuje zpracování přijatých rámců protokolu Spinel, zatímco další úloha se stará o periodické čtení teplotních senzorů. Podrobnější popis běhu jednotlivých úloh a zpracování komunikačních rámců je uveden v následujících sekcích.

6.1 Struktura firmware měřicího modulu

Firmware mikrokontroléru je rozdělen do několika logických částí. Základ tvoří hardwarově závislé části programu, které zajišťují obsluhu rozhraní UART a komunikaci s teplotními senzory TMP275. Další část programu zajišťuje ukládání konfigurace do interní paměti Flash a aplikační vrstva interpretuje příkazy přijaté prostřednictvím protokolu Spinel a vytváří odpovědi.

Důležitou část tvoří také správce zařízení, který uchovává informace o jednotlivých senzorech, jejich adrese, stavu připojení, poslední naměřené teplotě a dalších konfiguračních údajích. Tyto údaje jsou využívány jak při periodickém měření teploty, tak při zpracování požadavků přijatých z nadřazeného systému.

6.2 Inicializace systému

Inicializace systému probíhá v hlavní funkci programu po spuštění mikrokontroléru. Nejprve jsou zavolány inicializační funkce vývojové desky, které nastavují piny, hodinový systém a základní periférie mikrokontroléru. Následně je inicializováno rozhraní I^2C určené pro komunikaci s teplotními senzory TMP275.

Po inicializaci hardwarových prostředků je připravena vrstva persistentního úložiště. Program se pokusí inicializovat obsluhu interní paměti Flash a načíst dříve uloženou konfiguraci. Pokud je konfigurace k dispozici, jsou z ní převzaty zejména údaje o přenosové rychlosti rozhraní UART a konfigurační údaje jednotlivých zařízení.

Na základě načtené konfigurace je následně inicializováno rozhraní UART, které slouží pro komunikaci s nadřazeným počítačem. Poté jsou vytvořeny hlavní objekty programu, zejména správce zařízení, aplikační vrstva a správce protokolu. V závěru inicializace jsou vytvořeny úlohy operačního systému FreeRTOS a spuštěn plánovač.

6.3 Rozdělení programu do úloh FreeRTOS

Běh programu je rozdělen do samostatných úloh, které odpovídají hlavním funkcím měřicího modulu. Jedna úloha zajišťuje zpracování přijatých rámců protokolu Spinel, zatímco druhá úloha se stará o periodické čtení teplotních senzorů připojených ke sběrnici I^2C.

Úloha zajišťující zpracování protokolu přebírá dokončené rámce z přijímací fronty a předává je k dalšímu zpracování aplikační vrstvě. Na základě obsahu přijatého rámce je následně vytvořena odpověď, případně jsou provedeny změny konfiguračních údajů nebo jiné požadované operace.

Druhá úloha je určena pro periodické měření teploty. Po spuštění nejprve provede detekci dostupných senzorů a následně v pravidelných intervalech čte jejich aktuální hodnoty. Získaná data jsou ukládána do struktur spravovaných firmwarem, aby mohla být později využita při zpracování požadavků nadřazeného systému.

Kromě vlastních úloh využívá firmware také softwarový časovač operačního systému FreeRTOS. Ten slouží ke sledování časového limitu mezi přijatými bajty na sériové lince. Pokud během příjmu rámce dojde k překročení stanoveného intervalu, parser ukončí zpracování neúplného rámce a vrátí se do výchozího stavu. Tím je zajištěna odolnost komunikace proti přerušenému nebo neúplnému přenosu.

6.4 Příjem a zpracování komunikačních rámců

Příjem dat ze sériové linky je realizován pomocí přerušení rozhraní UART. Obslužná rutina přerušení reaguje na přijetí nového bajtu a předává jej parseru protokolu Spinel. Parser postupně skládá přijatý

rámec a vyhodnocuje jeho správnost. Pokud je rámec dokončen, je uložen do přijímací fronty, odkud jej převezme úloha správce protokolu.

Součástí obsluhy přerušení je také detekce chybových stavů rozhraní UART, například přetečení přijímače, chyby rámování nebo výskytu šumu na lince. V takovém případě obslužná rutina přerušení vyvolá návrat parseru do výchozího stavu a případný rozpracovaný rámec je zahozen. Tím je omezen vliv komunikačních chyb na další běh programu.

Vlastní zpracování přijatého rámce již probíhá mimo přerušení v kontextu samostatné úlohy. Správce protokolu předá přijatý rámec dalším vrstvám protokolu a následně aplikační vrstvě, která podle obsahu požadavku zajistí vytvoření odpovědi. Tento postup odděluje časově kritický příjem dat od náročnějšího zpracování příkazu a zjednodušuje návrh celé aplikace.

6.5 Obsluha teplotních senzorů TMP275

Obsluha teplotních senzorů je realizována samostatnou úlohou, která využívá sběrnici I^2C. Po spuštění programu je nejprve provedena detekce dostupných senzorů na předpokládaných adresách. U každého zařízení je ověřeno, zda na dané adrese odpovídá senzor TMP275 a zda je možné provést jeho základní inicializaci.

Pokud je senzor úspěšně detekován, jsou ve správci zařízení nastaveny příznaky odpovídající jeho připojení a správné funkci. Současně je uložena i první načtená hodnota teploty. V případě neúspěšné komunikace je zařízení označeno jako nepřipojené nebo chybové.

Po dokončení detekce přechází úloha do periodického režimu. V pravidelných časových intervalech postupně čte teplotu ze všech evidovaných senzorů, které jsou označeny jako připojené. Naměřené hodnoty jsou ukládány do datových struktur jednotlivých zařízení, odkud mohou být následně využity při zpracování požadavků nadřazeného systému.

Při čtení teploty jsou rozlišovány různé výsledky komunikace. Úspěšné přečtení hodnoty vede k aktualizaci poslední naměřené teploty a potvrzení správné funkce senzoru. Pokud senzor na dané adrese přestane odpovídat, je označen jako odpojený. Ostatní chyby komunikace jsou zaznamenány pomocí stavových příznaků, aniž by došlo k okamžitému odstranění zařízení ze seznamu aktivních senzorů.

6.6 Ukládání konfigurace do interní paměti Flash

Součástí firmware měřicího modulu je také mechanismus pro ukládání konfiguračních údajů do interní paměti Flash mikrokontroléru. Tato část programu slouží k zachování vybraných parametrů i po vypnutí napájení nebo restartu zařízení. Ukládány jsou zejména údaje související s komunikačním nastavením a konfigurací jednotlivých zařízení spravovaných aplikační vrstvou.

Konfigurační data jsou uložena ve speciální datové struktuře, která obsahuje identifikační hodnotu, sekvenční číslo, nastavenou přenosovou rychlost rozhraní UART a konfigurační údaje jednotli-

vých zařízení. Pro každé zařízení jsou uchovávány zejména jeho adresa v protokolu Spinel, informace o použití kontrolního součtu a uživatelská data.

Vlastní ukládání konfigurace je realizováno do dvou vyhrazených sektorů interní paměti Flash. Jednotlivé záznamy jsou zapisovány postupně do volných míst v aktivním sektoru. Při zaplnění sektoru je druhý sektor vymazán a nový záznam je uložen na jeho začátek. Pro výběr aktuálně platné konfigurace je využito sekvenční číslo, podle kterého je po spuštění systému vyhledán nejnovější platný záznam.

Při startu programu se firmware pokusí načíst poslední uloženou konfiguraci a použít ji při inicializaci systému. Při změně vybraných parametrů, například komunikační rychlosti nebo konfiguračních údajů zařízení, je následně vytvořen nový konfigurační záznam a uložen do paměti Flash. Tím je zajištěno zachování nastavení i po restartu zařízení.

6.7 Aplikační vrstva na straně mikrokontroléru

Aplikační vrstva na straně mikrokontroléru zajišťuje vlastní interpretaci příkazů přijatých prostřednictvím protokolu Spinel a vytváření odpovědí odesílaných nadřazenému systému. Její úlohou je propojit komunikační část programu s interní reprezentací jednotlivých zařízení, jejich stavů a naměřených hodnot.

Základ této vrstvy tvoří třída `Application`, která přijímá již zpracovaný rámec z nižších vrstev protokolu a podle jeho adresace a obsahu rozhoduje o dalším postupu. Pokud je rámec určen konkrétnímu zařízení, je vyhledána odpovídající položka ve správci zařízení. V případě broadcastu jsou požadavky postupně aplikovány na všechna evidovaná zařízení, aniž by byla odesílána odpověď.

Vlastní obsluha jednotlivých instrukcí je soustředěna do třídy `InstructionHandlers`. Ta obsahuje seznam podporovaných příkazů a k nim přiřazené obslužné funkce. Při zpracování rámce je podle typu instrukce vybrána odpovídající obsluha, která provede požadovanou operaci a připraví datový obsah odpovědi. Tímto způsobem jsou realizovány například příkazy pro čtení teploty, čtení surové hodnoty, změnu komunikačních parametrů, práci s uživatelskými daty nebo změnu stavových příznaků zařízení.

Důležitou součástí aplikační vrstvy je také kontrola oprávnění k provádění vybraných operací. Některé příkazy lze vykonat pouze v případě, že bylo předtím povoleno konfigurační nastavení zařízení. Tím je oddělena běžná provozní komunikace od příkazů, které mění nastavení modulu.

Po úspěšném zpracování příkazu může aplikační vrstva vyvolat také následné akce, například uložení nové konfigurace do interní paměti Flash nebo změnu přenosové rychlosti sériové linky. Tím je zajištěno propojení mezi logikou protokolu, stavem zařízení a uloženou konfigurací modulu.

Přehled implementovaných instrukcí je uveden v tabulce 6.1.

Tabulka 6.1: Přehled implementovaných instrukcí aplikační vrstvy

Instrukce	**Význam**
GET_TEMP	Vrácení aktuální hodnoty teploty
GET_RAW	Vrácení surové hodnoty teplotního registru senzoru
RESET	Vyvolání restartu zařízení
SET_STATUS	Nastavení stavového bajtu zařízení
SET_CONFIG	Povolení konfiguračního režimu zařízení
SET_COMMS	Nastavení adresy a komunikační rychlosti
SET_ADDR	Nastavení adresy zařízení v ASCII režimu
SET_SPEED	Nastavení komunikační rychlosti v ASCII režimu
READ_COMMS	Čtení adresy a nastavené komunikační rychlosti
READ_ERRORS	Čtení počtu komunikačních chyb
READ_STATUS	Čtení stavového bajtu zařízení
WRITE_USER_DATA	Zápis uživatelských dat
READ_USER_DATA	Čtení uživatelských dat zařízení
READ_INFO	Čtení seznamu dostupných zařízení a jejich adres
READ_CHECKSUM	Čtení informace o použití kontrolního součtu
SET_CHECKSUM	Povolení nebo zakázání kontrolního součtu
READ_PROD_INFO	Čtení výrobních údajů zařízení

6.8 Aplikační vrstva na straně nadřazeného počítače

Na straně nadřazeného počítače byla vytvořena jednoduchá testovací aplikace, která sloužila k ověřování implementace protokolu Spinel i k praktickému testování výsledného měřicího modulu. Aplikace umožňuje otevírat sériový port, sestavovat a odesílat požadavky v ASCII i binární podobě a zpracovávat odpovědi přijaté od zařízení.

Program využívá stejné základní části protokolu jako firmware mikrokontroléru, zejména parser, správce rámců, správce protokolu a aplikační vrstvu pro vytváření požadavků a vyhodnocování odpovědí. Díky tomu bylo možné jednotným způsobem testovat komunikaci na obou stranách přenosu.

Aplikace podporuje dva režimy činnosti. V monitorovacím režimu po spuštění automaticky detekuje dostupná zařízení na lince a následně periodicky čte jejich teplotu. V ladicím režimu je možné ručně vyvolávat jednotlivé instrukce protokolu, například čtení teploty, čtení stavových a uživatelských dat, změnu komunikačních parametrů, reset zařízení nebo práci s kontrolním součtem.

Součástí aplikace je také kontrola časového limitu odpovědi zařízení. Pokud po odeslání požadavku nedojde v očekávaném čase k přijetí odpovědi, je situace vyhodnocena jako timeout. Tato vlastnost byla využita při ověřování komunikace i při testování vybraných chybových stavů.

Testovací aplikace tak sloužila jako základní nástroj pro oživování, ladění a výsledné ověřování celého systému.

6.9 Shrnutí kapitoly

Kapitola popsala implementaci firmware měřicího modulu i základní principy aplikační vrstvy na straně nadřazeného počítače. Bylo ukázáno rozdělení programu do jednotlivých logických částí, způsob inicializace systému, využití úloh operačního systému FreeRTOS, zpracování komunikačních rámců, obsluha teplotních senzorů TMP275 a mechanismus ukládání konfigurace do interní paměti Flash.

Součástí kapitoly byl také popis aplikační vrstvy na straně mikrokontroléru i nadřazeného počítače. Na straně mikrokontroléru tato vrstva zajišťuje interpretaci instrukcí protokolu Spinel a vytváření odpovědí, zatímco na straně počítače umožňuje sestavování požadavků, příjem odpovědí a jejich vyhodnocení při testování. Tím je vytvořen programový základ pro funkci navrženého měřicího modulu i pro jeho následné ověřování v praktickém provozu.

Kapitola 7

Ověření funkce navrženého řešení

Kapitola se zabývá ověřením funkce navrženého měřicího modulu a jednotlivých částí jeho programového i hardwarového řešení. Cílem je ukázat, jakým způsobem probíhalo oživování systému, jak byly ověřovány jednotlivé části implementace a jaké výsledky byly při testování dosaženy.

Ověřování probíhalo postupně od samostatného zkoumání protokolu Spinel na referenčním zařízení až po testování vlastního firmwaru na vývojové desce FRDM-MCXN947. Pozornost byla věnována zejména správné interpretaci instrukcí protokolu, komunikaci mezi mikrokontrolérem a nadřazeným počítačem, měření teploty prostřednictvím senzorů TMP275 a ukládání konfiguračních údajů do interní paměti Flash.

7.1 Postup oživování a ověřování systému

Ověřování navrženého řešení probíhalo postupně v několika krocích. Nejprve byla pozornost věnována samotnému protokolu Spinel a způsobu jeho komunikace. Pro tento účel bylo využito referenční zařízení společnosti Papouch, které bylo připojeno k osobnímu počítači prostřednictvím rozhraní RS485. Pomocí terminálu bylo možné ručně zadávat ASCII příkazy a sledovat chování zařízení při různých typech požadavků. Tímto způsobem bylo možné ověřit základní pravidla komunikace, strukturu odpovědí a reakce zařízení na různé situace.

Po zvládnutí základů protokolu byla vytvořena jednoduchá aplikace na straně osobního počítače, která umožnila sestavování a odesílání požadavků nejen v ASCII podobě, ale také v binární formě. Tato aplikace byla následně využívána jak při ověřování komunikace s referenčním zařízením, tak později při testování vlastního měřicího modulu. Současně umožnila ověřit správné rozebírání přijatých odpovědí a oddělení vlastních dat od obalových částí komunikačního rámce.

Dalším krokem bylo oživení vlastního mikrokontroléru, nejprve z hlediska komunikace přes rozhraní UART. Po správném nastavení periferií a zapojení komunikační linky bylo ověřeno vysílání a příjem dat mezi mikrokontrolérem a nadřazeným počítačem. Následně byla do programu doplněna

základní aplikační logika, takže modul začal reagovat na vybrané požadavky nejprve zkušebními odpověďmi a později reálnými daty.

Po ověření základní komunikace byl firmware rozšířen o běh pod operačním systémem FreeRTOS a o periodické měření teploty prostřednictvím sběrnice I^2C. Při oživování komunikace se senzorem TMP275 se objevily problémy, které byly nakonec odstraněny úpravou konfigurace příslušné periférie mikrokontroléru. Po zprovoznění měření teploty byl doplněn mechanismus ukládání konfigurace do interní paměti Flash. V závěrečné fázi pak probíhalo průběžné testování všech implementovaných instrukcí a ladění detailů výsledného řešení.

7.2 Ověření implementace protokolu Spinel

Zvláštní pozornost byla věnována správné funkci implementace protokolu Spinel. Testování probíhalo nejprve na referenčním zařízení společnosti Papouch a následně i na vlastním měřicím modulu.

Pro ověřování byla využívána jednoduchá aplikace na straně osobního počítače, která umožnila sestavování a odesílání požadavků v ASCII i binární podobě. Tím bylo možné ověřit nejen textovou variantu komunikace, ale i binární přenos rámců, který není možné pohodlně testovat běžným terminálem. Současně byla ověřena správná interpretace přijatých odpovědí a oddělení vlastních dat od obalových částí komunikačního rámce.

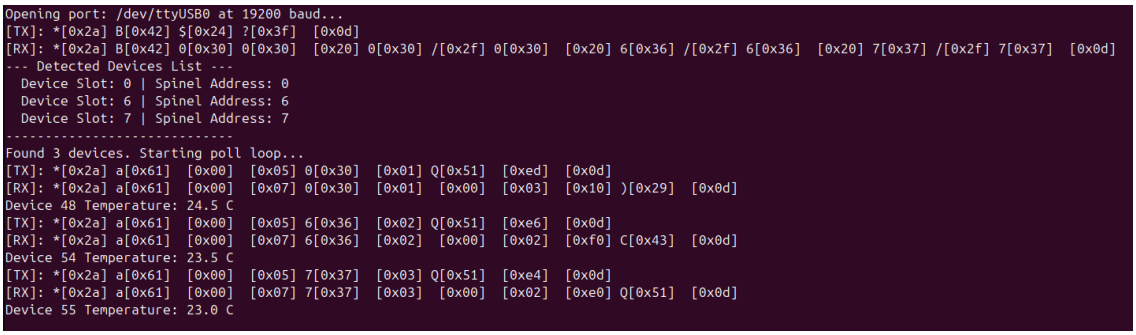
Po přenesení implementace na vlastní mikrokontrolér byla správná funkce protokolu ověřována pomocí testovací aplikace na osobním počítači komunikující přes převodník RS485. Na modulu byly postupně vyzkoušeny všechny aktuálně implementované instrukce uvedené v tabulce 6.1. U instrukcí podporujících ASCII režim byla ověřena jejich textová varianta, zatímco u instrukcí podporujících binární režim byla ověřena binární forma přenosu.

Ověřením těchto funkcí byla potvrzena správná interpretace požadavků, vytváření odpovědí i návaznost aplikační vrstvy na nižší části implementace protokolu Spinel.

7.3 Ověření komunikace mikrokontroléru s nadřazeným počítačem

Komunikace mezi mikrokontrolérem a nadřazeným počítačem byla ověřována v zapojení tvořeném testovací aplikací na osobním počítači, převodníkem USB/RS485 a výsledným měřicím modulem realizovaným na desce FRDM-MCXN947. Přenos dat mezi mikrokontrolérem a nadřazeným počítačem probíhal prostřednictvím rozhraní UART a sběrnice RS485. Tím bylo možné sledovat správnost přenosu dat v obou směrech a současně ověřit spolupráci mezi komunikační vrstvou, parserem a aplikační logikou měřicího modulu.

Ukázka průběhu navázání komunikace mezi testovací aplikací a měřicím modulem je uvedena na obrázku 7.1. Po otevření sériového portu aplikace provedla detekci dostupných zařízení na lince a následně zahájila periodické čtení teploty. Ve výpisu jsou současně zobrazeny odesílané a přijaté rámce i dekódované hodnoty teplot jednotlivých zařízení.



Obrázek 7.1: Ukázka navázání komunikace testovací aplikace s měřicím modulem, detekce dostupných zařízení a zahájení periodického čtení teploty

V průběhu testování bylo ověřeno, že mikrokontrolér správně přijímá požadavky přicházející z nadřazeného počítače, rozpoznává jejich obsah a vytváří odpovídající odpovědi. Současně bylo potvrzeno, že odesílané odpovědi jsou přenášeny ve správném formátu a mohou být na straně osobního počítače bez problémů zpracovány. Ověření proběhlo jak pro ASCII komunikaci, tak pro binární přenos rámců.

Pozornost byla věnována také chování systému při změně komunikačních parametrů. Byla ověřena správná reakce modulu na požadavky související se změnou adresy zařízení a nastavením přenosové rychlosti. Tím bylo potvrzeno, že komunikační část firmware správně spolupracuje s aplikační vrstvou i s mechanismem ukládání konfigurace.

Výsledky testování potvrdily, že navržený měřicí modul je schopen spolehlivě komunikovat s nadřazeným počítačem prostřednictvím sběrnice RS485 a správně zpracovávat požadavky přenášené protokolem Spinel.

7.4 Ověření měření teploty pomocí senzoru TMP275

Měření teploty bylo v praktické části ověřováno s více připojenými senzory TMP275. Po zprovoznění komunikace po sběrnici I^2C byla nejprve ověřena správná detekce senzorů na očekávaných adresách a následně i jejich inicializace v programu mikrokontroléru.

Po úspěšné detekci senzorů probíhalo periodické čtení teploty v samostatné úloze operačního systému FreeRTOS. Naměřené hodnoty byly ukládány do datových struktur spravovaných firmwarem a následně zpřístupněny aplikační vrstvě. Tím bylo možné ověřit návaznost mezi obsluhou senzorů, správou interního stavu zařízení a komunikací s nadřazeným počítačem.

V rámci testování byly ověřeny zejména instrukce určené pro čtení teplotních údajů, tedy `GET_TEMP` a `GET_RAW`. Bylo potvrzeno, že modul vrací jak přepočtenou hodnotu teploty, tak i surové hodnoty teplotních registrů jednotlivých senzorů. Tím byla ověřena správná funkce jak samotného měření, tak i následného zpracování dat ve firmware.

Výsledky testování potvrdily, že po správné inicializaci rozhraní I^2C je měřicí modul schopen spolehlivě komunikovat se senzory TMP275 a pravidelně získávat jejich aktuální hodnoty teploty.

7.5 Ověření ukládání konfigurace do interní paměti Flash

Ukládání konfigurace do interní paměti Flash bylo ověřováno při změnách parametrů, které mají být po restartu zařízení zachovány. Pozornost byla věnována zejména komunikačnímu nastavení a dalším konfiguračním údajům spravovaným aplikační vrstvou.

V průběhu testování bylo ověřeno, že po změně vybraných parametrů dojde k vytvoření nového konfiguračního záznamu a jeho uložení do interní paměti Flash. Současně byla sledována správná návaznost mezi aplikační vrstvou, mechanismem ukládání a následným načtením konfigurace při startu systému.

Byla ověřena také správná práce s konfigurační strukturou obsahující identifikační hodnotu, sekvenční číslo, přenosovou rychlost rozhraní UART a údaje jednotlivých zařízení. Tím bylo potvrzeno, že firmware ukládá pouze potřebná data a že je možné po spuštění systému vyhledat nejnovější platný záznam.

Testování potvrdilo, že firmware správně vytváří a ukládá nové konfigurační záznamy, které mohou být následně využity při dalším spuštění systému.

7.6 Shrnutí kapitoly

Kapitola popsala postup oživování a ověřování navrženého řešení od prvotního zkoumání protokolu Spinel na referenčním zařízení až po testování vlastního měřicího modulu. Byla ověřena správná funkce implementace protokolu, komunikace mezi mikrokontrolérem a nadřazeným počítačem, měření teploty pomocí senzorů TMP275 i mechanismus ukládání konfigurace do interní paměti Flash.

Výsledky testování potvrdily, že navržený měřicí modul je schopen správně zpracovávat požadavky přenášené protokolem Spinel, komunikovat s nadřazeným systémem prostřednictvím sběrnice RS485 a získávat aktuální hodnoty teploty ze senzorů TMP275. Současně byla ověřena i funkce ukládání konfiguračních údajů, která tvoří důležitou součást výsledného firmware.

Kapitola 8

Závěr

Cílem bakalářské práce bylo seznámit se s komunikačním protokolem Spinel, navrhnout jeho softwarovou implementaci a vytvořit měřicí modul schopný komunikace s nadřazeným počítačem prostřednictvím sběrnice RS485 a současně zajišťovat měření teploty pomocí digitálních senzorů připojených po sběrnici I^2C.

Nejprve byl popsán protokol Spinel, jeho struktura a princip komunikace. Na tento rozbor následně navázal návrh softwarové implementace protokolu a vytvoření jednotlivých částí programu potřebných pro zpracování přijatých rámců, interpretaci instrukcí a vytváření odpovědí. Současně byl navržen a realizován měřicí modul založený na vývojové desce FRDM-MCXN947, převodníku MAX485 a teplotním senzoru TMP275.

Významnou část práce tvořila implementace firmware měřicího modulu v prostředí FreeRTOS. Firmware zajišťuje příjem a zpracování komunikačních rámců, periodické měření teploty, správu aplikačních údajů jednotlivých zařízení a ukládání konfigurace do interní paměti Flash mikrokontroléru. Součástí výsledného řešení je také aplikační vrstva na straně nadřazeného počítače určená pro testování a ověřování komunikace.

Navržené řešení bylo následně prakticky ověřeno. Byla potvrzena správná funkce implementace protokolu Spinel, komunikace mezi mikrokontrolérem a nadřazeným počítačem prostřednictvím sběrnice RS485, měření teploty pomocí senzoru TMP275 i ukládání konfigurace do interní paměti Flash. Testování ukázalo, že navržený měřicí modul je schopen plnit požadované funkce a že vytvořené řešení odpovídá zadání práce.

Navržený systém současně představuje vhodný základ pro případné další rozšiřování, například z hlediska připojení většího počtu senzorů nebo doplnění dalších typů měřených veličin. V současném stavu však již tvoří funkční implementaci měřicího modulu komunikujícího prostřednictvím protokolu Spinel.

Literatura

1. PAPOUCH S.R.O. *Spinel - obecný popis* [online]. 2016. [cit. 2026-02-28]. Dostupné z: `https://cdn.papouch.com/data/user-content/old_eshop/files_article/Spinel/spinel_obecne.pdf`.
2. PAPOUCH S.R.O. *Spinel - implementace komunikačního protokolu* [online]. 2016. [cit. 2026-02-28]. Dostupné z: `https://cdn.papouch.com/data/user-content/old_eshop/files_article/Spinel/spinel_implementace.pdf`.
3. NXP B.V. *FRDM-MCXN947 Development Board: Quick Start Guide*. NXP, 2024. Document Number: MCXN947QSG, Rev. 1.
4. NXP SEMICONDUCTORS. *MCXNP184M150F70* [online]. [cit. 2026-04-13]. Dostupné z: `https://www.nxp.com/docs/en/data-sheet/MCXNP184M150F70.pdf`.
5. MAXIM INTEGRATED. *MAX485* [online]. [cit. 2026-04-13]. Dostupné z: `https://eu.mouser.com/datasheet/3/1014/1/MAX1487E_MAX491E.pdf`.
6. NXP SEMICONDUCTORS. *UM10204 I2C-bus specification and user manual* [online]. 2021. [cit. 2026-04-13]. Dostupné z: `https://www.nxp.com/docs/en/user-guide/UM10204.pdf`.
7. TEXAS INSTRUMENTS. *TMP275-Q1* [online]. [cit. 2026-04-13]. Dostupné z: `https://www.ti.com/lit/ds/symlink/tmp275-q1.pdf`.

Příloha A

Ukázky úspěšně provedených testů komunikace

V této příloze jsou uvedeny doplňující ukázky úspěšně provedených testů komunikace mezi testovací aplikací na počítači a měřicím modulem. Jednotlivé výpisy dokumentují správné zpracování vybraných implementovaných instrukcí protokolu Spinel a odpovídající reakce zařízení. Ukázky zahrnují čtení teplotních a diagnostických údajů, práci se stavovými a uživatelskými daty i vybrané konfigurační a servisní operace.

A.1 Čtení provozních a diagnostických údajů

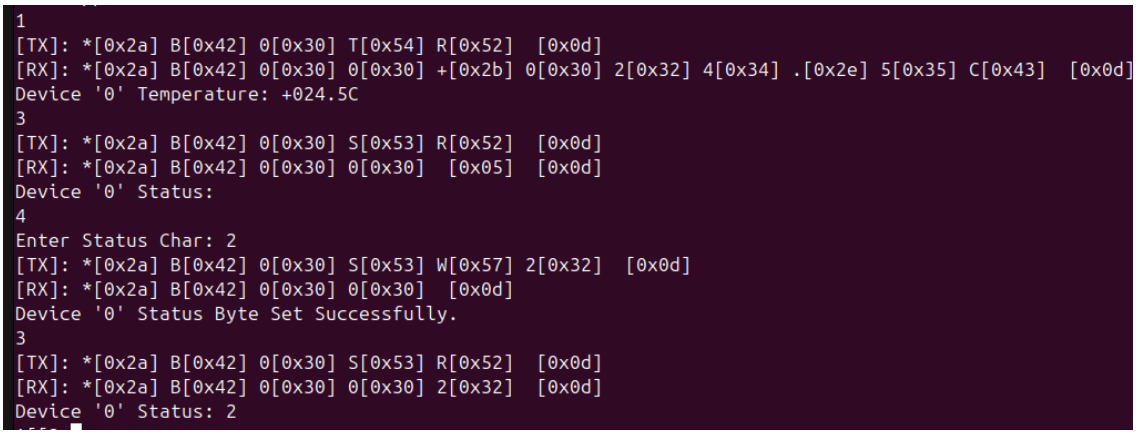
Na obrázku A.1 je uvedena ukázka úspěšného provedení instrukcí `GET_RAW`, `READ_COMMS`, `READ_PROD_INFO` a `READ_ERRORS`. Výpis dokládá správné čtení surové hodnoty teplotního registru, komunikačních parametrů zařízení, výrobních informací a počtu zaznamenaných komunikačních chyb.

```
2
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x04] _[0x5f]  [0xda]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x07] 2[0x32]  [0x04]  [0x00]  [0x19]  [0x80]  [0x9e]  [0x0d]
Device 50 Sensor RAW Value: 6528 (0x1980)
0
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x05]  [0xf0] H[0x48]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x07] 2[0x32]  [0x05]  [0x00] 2[0x32]  [0x07]  [0xfd]  [0x0d]
Device 50 Params - Address: 2, Speed: 19200
p
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x06]  [0xfa] =[0x3d]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x0d]
2[0x32]  [0x06]  [0x00] B[0x42] A[0x41] K[0x4b]  [0x20] 2[0x32] 0[0x30] 2[0x32] 6[0x36] w[0x77]  [0x0d]
=== Device 50 Production Info ===
Info: BAK 2026
========================
e
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x07]  [0xf4] B[0x42]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x06] 2[0x32]  [0x07]  [0x00]  [0x00] 5[0x35]  [0x0d]
Device 50 Communication Errors: 0
```

Obrázek A.1: Ukázka čtení provozních a diagnostických údajů zařízení

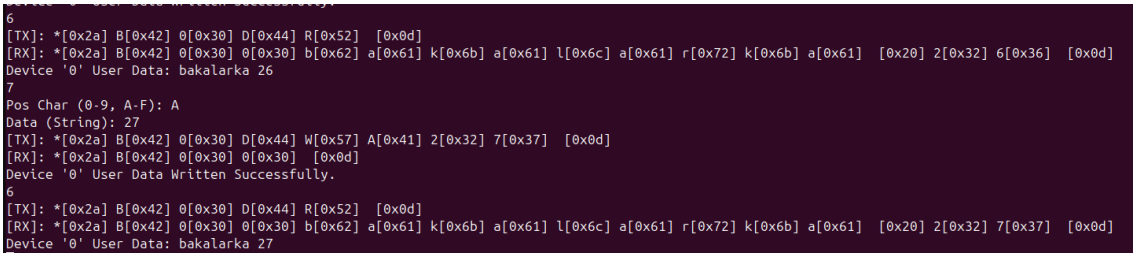
A.2 Práce se stavovými a uživatelskými daty

Na obrázku A.2 je zachycena ukázka čtení a zápisu stavového bajtu zařízení pomocí instrukcí `READ_STATUS` a `SET_STATUS`. Výpis potvrzuje, že po zápisu nové hodnoty bylo jejím opětovným čtením ověřeno správné uložení změny.



Obrázek A.2: Ukázka čtení a zápisu stavového bajtu zařízení

Na obrázku A.3 je uvedena ukázka čtení a zápisu uživatelských dat pomocí instrukcí `READ_USER_DATA` a `WRITE_USER_DATA`. Po zápisu nové hodnoty byla správnost operace ověřena jejím následným opětovným přečtením.



Obrázek A.3: Ukázka čtení a zápisu uživatelských dat zařízení

A.3 Konfigurační a servisní instrukce

Na obrázku A.4 je uvedena ukázka čtení a změny nastavení kontrolního součtu pomocí instrukcí `READ_CHECKSUM` a `SET_CHECKSUM`. Výpis potvrzuje, že po změně nastavení byla nová hodnota správně vrácena při následném čtení.

Na obrázku A.5 je zachycena ukázka instrukce `RESET`. Před resetem bylo pomocí instrukce `READ_INFO` potvrzeno, že jsou na lince dostupná dvě zařízení. Po provedení resetu byla stejným způsobem ověřena dostupnost tří zařízení.

```
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x08]  [0xfe] 7[0x37]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x06] 2[0x32]  [0x08]  [0x00]  [0x00] 4[0x34]  [0x0d]
Device 50 Checksum: NOT ENABLED
d
Enable checksum? (y/n): y
[TX]: *[0x2a] a[0x61]  [0x00]  [0x06] 2[0x32]  [0x09]  [0xee]  [0x01] D[0x44]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x09]  [0x00] 4[0x34]  [0x0d]
Device 50 CRC set correctly
c
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x0a]  [0xfe] 5[0x35]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x06] 2[0x32]  [0x0a]  [0x00]  [0x01] 1[0x31]  [0x0d]
Device 50 Checksum: ENABLED
```

Obrázek A.4: Ukázka čtení a změny nastavení kontrolního součtu

```
5
[TX]: *[0x2a] B[0x42] 0[0x30] ?[0x3f]  [0x0d]
[RX]: *[0x2a] B[0x42] 0[0x30] 0[0x30]  [0x20] 0[0x30] /[0x2f] 0[0x30]  [0x20] 6[0x36] /[0x2f] 6[0x36]  [0x0d]
--- Detected Devices List ---
  Device Slot: 0 | Spinel Address: 0
  Device Slot: 6 | Spinel Address: 6
---------------------------
r
[TX]: *[0x2a] B[0x42] 0[0x30] R[0x52] E[0x45]  [0x0d]
[RX]: *[0x2a] B[0x42] 0[0x30] 0[0x30]  [0x0d]
Device '0' Reset Command Acknowledged.
5
[TX]: *[0x2a] B[0x42] 0[0x30] ?[0x3f]  [0x0d]
[RX]: *[0x2a] B[0x42] 0[0x30] 0[0x30]  [0x20] 0[0x30] /[0x2f] 0[0x30]  [0x20] 6[0x36] /[0x2f] 6[0x36]  [0x20] 7[0x37] /[0x2f] 7[0x37]  [0x0d]
--- Detected Devices List ---
  Device Slot: 0 | Spinel Address: 0
  Device Slot: 6 | Spinel Address: 6
  Device Slot: 7 | Spinel Address: 7
---------------------------
```

Obrázek A.5: Ukázka resetu zařízení a následného ověření dostupných zařízení na lince

Na obrázku A.6 je uvedena ukázka povolení konfigurace, změny adresy zařízení a následného ověření nové adresy pomocí opětovné detekce zařízení na lince.

Na obrázku A.7 je uvedena ukázka změny přenosové rychlosti zařízení a následného ověření nové hodnoty pomocí čtení komunikačních parametrů.

```
5
[TX]: *[0x2a] B[0x42] 0[0x30] ?[0x3f]  [0x0d]
[RX]: *[0x2a] B[0x42] 0[0x30] 0[0x30]  [0x20] 0[0x30] /[0x2f] 0[0x30]  [0x20] 6[0x36] /[0x2f] 6[0x36]  [0x20] 7[0x37] /[0x2f] 7[0x37]  [0x0d]
--- Detected Devices List ---
  Device Slot: 0 | Spinel Address: 0
  Device Slot: 6 | Spinel Address: 6
  Device Slot: 7 | Spinel Address: 7
----------------------------
8
[TX]: *[0x2a] B[0x42] 0[0x30] E[0x45]  [0x0d]
[RX]: *[0x2a] B[0x42] 0[0x30] 0[0x30]  [0x0d]
Device '0' Configuration Enabled.
9
New Addr Char: 2
[TX]: *[0x2a] B[0x42] 0[0x30] A[0x41] S[0x53] 2[0x32]  [0x0d]
[RX]: *[0x2a] B[0x42] 2[0x32] 0[0x30]  [0x0d]
Device '2' Communication Settings Changed.
5
[TX]: *[0x2a] B[0x42] 0[0x30] ?[0x3f]  [0x0d]
TIMEOUT: Device 48 did not answer in time.
t
New Ascii Addr: 2
5
[TX]: *[0x2a] B[0x42] 2[0x32] ?[0x3f]  [0x0d]
[RX]: *[0x2a] B[0x42] 2[0x32] 0[0x30]  [0x20] 0[0x30] /[0x2f] 2[0x32]  [0x20] 6[0x36] /[0x2f] 6[0x36]  [0x20] 7[0x37] /[0x2f] 7[0x37]  [0x0d]
--- Detected Devices List ---
  Device Slot: 0 | Spinel Address: 2
  Device Slot: 6 | Spinel Address: 6
  Device Slot: 7 | Spinel Address: 7
----------------------------
```

Obrázek A.6: Ukázka povolení konfigurace, změny adresy zařízení a ověření nové adresy

```
0
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x02]  [0xf0] K[0x4b]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x07] 2[0x32]  [0x02]  [0x00] 2[0x32]  [0x07]  [0x00]  [0x0d]
Device 50 Params - Address: 2, Speed: 19200
8
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x03]  [0xe4] V[0x56]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x03]  [0x00] :[0x3a]  [0x0d]
Device 50 Configuration Enabled.
9
New Addr (0-253): 50
Baud (e.g. 9600): 9600
[TX]: *[0x2a] a[0x61]  [0x00]  [0x07] 2[0x32]  [0x04]  [0xe0] 2[0x32]  [0x06]  [0x1f]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x04]  [0x00] 9[0x39]  [0x0d]
Device 50 Communication Settings Changed.
0
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 2[0x32]  [0x05]  [0xf0] H[0x48]  [0x0d]
[RX]: *[0x2a] a[0x61]  [0x00]  [0x07] 2[0x32]  [0x05]  [0x00] 2[0x32]  [0x06]  [0xfe]  [0x0d]
Device 50 Params - Address: 2, Speed: 9600
```

Obrázek A.7: Ukázka změny přenosové rychlosti a ověření nové hodnoty

Příloha B

Ukázky chybových a hraničních stavů komunikace

V této příloze jsou uvedeny doplňující ukázky chybových a hraničních stavů komunikace mezi testovací aplikací na počítači a měřicím modulem. Jednotlivé výstupy dokumentují reakce zařízení na neplatné požadavky a ověřují správné chování implementace při zpracování chybových situací.

B.1 Chybové odpovědi zařízení

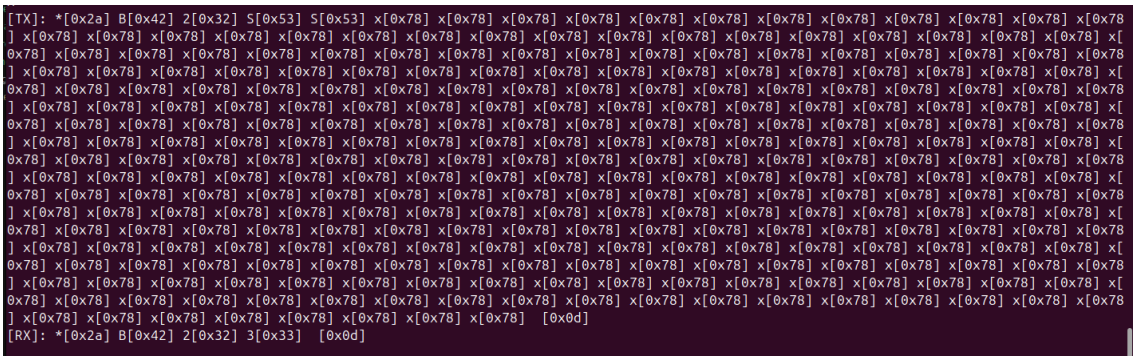
Na obrázku B.1 jsou zachyceny ukázky několika chybových stavů vyvolaných při komunikaci se zařízením. Ve výpisu je doložena reakce na neplatnou instrukci `INVALID_INSTRUCTION`, neplatná data `INVALID_DATA`, nepovolený zápis bez předchozího povolení konfigurace `UNAUTHORIZED_WRITE`, požadavek na neexistující adresu vyhodnocený na straně testovací aplikace jako timeout a také stav `DEVICE_FAILURE`, který odpovídá chybě zařízení nebo nedostupnému senzoru.

```
[TX]: *[0x2a] B[0x42] 2[0x32] t[0x74] r[0x72]  [0x0d]
[RX]: *[0x2a] B[0x42] 2[0x32] 2[0x32]  [0x0d]
Error [GET_TEMP] from Device '2': MCU answered Invalid Instruction
[TX]: *[0x2a] B[0x42] 2[0x32] S[0x53] W[0x57]  [0x0d]
[RX]: *[0x2a] B[0x42] 2[0x32] 3[0x33]  [0x0d]
Error [SET_STATUS] from Device '2': MCU answered Invalid Data
[TX]: *[0x2a] B[0x42] 2[0x32] S[0x53] S[0x53]  [0x0d]
[RX]: *[0x2a] B[0x42] 2[0x32] 4[0x34]  [0x0d]
Error [SET_COMM] from Device '2': MCU answered Unauthorized Write
[TX]: *[0x2a] a[0x61]  [0x00]  [0x05] 0[0x30]  [0x01] Q[0x51]  [0xed]  [0x0d]
TIMEOUT: Device 48 did not answer in time.
[TX]: *[0x2a] B[0x42] 1[0x31] S[0x53] W[0x57]  [0x0d]
[RX]: *[0x2a] B[0x42] 1[0x31] 5[0x35]  [0x0d]
Error [SET_STATUS] from Device '1': MCU answered Device Failure (Hardware error/Unavailable)
```

Obrázek B.1: Ukázka vybraných chybových stavů komunikace

B.2 Příliš dlouhý rámec

Na obrázku je B.2 zachycena ukázka testu s nadměrně dlouhým rámcem. Výpis není vzhledem k délce odesílaných dat zcela přehledný, avšak v odpovědi zařízení je vidět potvrzovací kód `3`, který odpovídá chybě `INVALID_DATA`. Tím byla potvrzena správná reakce systému na neplatně dlouhá data požadavku.



Obrázek B.2: Ukázka vybraných chybových stavů komunikace

B.3 Další ošetřené hraniční stavy

Některé další hraniční stavy, zejména neúplný rámec a rámec s chybným kontrolním součtem, jsou v implementaci rovněž ošetřeny. V případě neúplného rámce parser po překročení časového limitu ukončí jeho zpracování a vrátí se do výchozího stavu. Při chybě kontrolního součtu zařízení vrací chybovou odpověď odpovídající stavu `INVALID_DATA`.